

---

BACHELOR OF SCIENCE IN INFORMATION TECHNOLOGY

# ANALYSIS OF ALGORITHMS

## Correctness, Cost, and the Limits of Efficiency

---

### UNIFIED LECTURE HANDOUT

Complete Course in One Volume • Fall 2026



Two questions about every algorithm: is it right, and what does it cost?

*And a third the first two do not answer: is there a better one, and how would you know?*

**What this volume covers.** Every clause of the HEC 2023 outline for *Analysis of Algorithms*, in the order the outline gives them, with the published learning outcomes traced chapter by chapter in the Course Specification overleaf.

Under HEC 2023 this is a **computing core** course — compulsory in all ten BS computing degrees, this one included — taken in Semester 7, worth 3 (3–0) credit hours, with *Data Structures* as its prerequisite. The 2023 outline is word for word the 2017 outline, under which the course was BSCS-only and absent from BSIT.

About a third of that outline you have already met in Data Structures. Appendix F draws the line: that course owns the structures, this one owns the *analysis* and the design paradigms.

Six Parts • Twenty-Three Chapters • Seven Appendices

## How to Use This Handout

---

Most of what this course teaches is not an algorithm. It is a way of arguing about one.

By Semester 7 you can already write a program that sorts. What you cannot yet do reliably is say *why* it is correct for every input rather than the ones you tried, *what* it will cost when the input is a thousand times larger, and *whether* anything could do better. Those three questions — correctness, cost, and the limit — are the whole subject, and each has a technique behind it: the loop invariant, the asymptotic bound, and the lower bound.

### The six parts

---

**Part I — Analysing an Algorithm** (Chapters 1–4) builds the vocabulary. What “efficient” means and against what model; best, expected and worst case; the five asymptotic notations defined properly, not just  $O$ ; and the loop invariant, which is how a correctness claim is proved rather than asserted.

**Part II — Recursion and Divide-and-Conquer** (Chapters 5–9) is the machinery. Recurrences solved three ways, the master theorem and where it does not apply, brute force taken seriously as a baseline, then merge sort and quicksort analysed in full.

**Part III — Sorting, Selection and the Structures Beneath** (Chapters 10–13) establishes the first lower bound in the book — no comparison sort can beat  $n \lg n$  — and then shows what buying extra assumptions gets you. Chapters 12 and 13 revisit heaps, search trees and hashing from Data Structures, for their *analysis* rather than their implementation.

**Part IV — The Design Paradigms** (Chapters 14–17) is the part you will use most after the examination: greedy, dynamic programming, and the amortised analysis without which several earlier claims cannot be stated honestly.

**Part V — Graph Algorithms** (Chapters 18–20) applies all of it, and is where sparsity starts changing every bound you write.

**Part VI — Strings, Limits and Practice** (Chapters 21–23) closes with string matching, the complexity classes, and your project.



Why the lower bound in Chapter 10 matters more than the algorithms around it. Everything before it answers “how fast is this?”. Chapter 10 answers “how fast could *anything* be?”, and that is a different kind of statement — it is about the problem, not about any program. Once you have seen one proved, Chapter 22’s claim that we do not know whether  $P = NP$  stops being a slogan and becomes a recognisable open question of the same shape.

### The furniture of every chapter

---

| Box                | What it is for                                                         |
|--------------------|------------------------------------------------------------------------|
| 🎯 Outcomes         | What you will be able to <i>do</i> . Bloom verbs, not topics.          |
| 🔗 Before you start | Which earlier chapters this one stands on.                             |
| 📖 Key terms        | The vocabulary introduced; all of it is in the glossary of Appendix D. |

| Box                                                                                                                                                                                                                                                                                                       | What it is for                                                                                                                                                             |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  <b>Definition</b>                                                                                                                                                                                                       | The canonical statement. In this subject the definitions are load-bearing: most wrong answers in the examination are a misremembered definition, not a failed calculation. |
|  <b>Theorem</b> ,  <b>Lemma</b> ,<br> <b>Corollary</b> | A result, numbered within the chapter so the text can say “by Lemma 6.2”.                                                                                                  |
| <i>Proof.</i>                                                                                                                                                                                                                                                                                             | Deliberately not in a box — it is prose and should be read as prose. Every proof here is one you could reproduce; the ones that are not are cited instead.                 |
|  <b>Algorithm</b>                                                                                                                                                                                                        | Pseudocode, numbered by line, so the analysis can point at line 4.                                                                                                         |
|  <b>Concept</b>                                                                                                                                                                                                          | The judgement call — what an experienced practitioner would decide, and why.                                                                                               |
|  <b>Warning</b>                                                                                                                                                                                                          | Something that will bite you, usually in the examination.                                                                                                                  |
|  <b>Worked example</b>                                                                                                                                                                                                   | Real arithmetic, every step shown. Do them with a pen.                                                                                                                     |
|  <b>Common pitfalls</b>                                                                                                                                                                                                  | The mistakes that are actually made.                                                                                                                                       |
|  <b>Try it yourself</b>                                                                                                                                                                                                  | The laboratory: C++ you compile, run and <i>time</i> .                                                                                                                     |
|  <b>Checkpoint</b>                                                                                                                                                                                                       | Three questions. Attempt them before reading Appendix C.                                                                                                                   |
|  <b>Chapter summary</b>                                                                                                                                                                                                  | What to retain.                                                                                                                                                            |
|  <b>Review questions</b>                                                                                                                                                                                                | Examination practice: multiple choice, short answer, applied.                                                                                                              |

### The measured strand, and why it is here

Every chapter also carries at least one *measurement*: the algorithm compiled, run, and timed, with the figures printed in the book.

This is not decoration. Asymptotic analysis deliberately discards constant factors, and a student who only ever sees the asymptotics draws two wrong conclusions — that an  $O(n \lg n)$  algorithm always beats an  $O(n^2)$  one, and that the analysis is therefore unreliable. Both are cured by seeing where the crossover actually falls. Insertion sort really does beat merge sort on small inputs, and the asymptotics really do win by the time it matters.

Every figure in those boxes was produced by the file named beside it, in `code/`, with the compiler and flags recorded in Appendix A. Rerun them. Your numbers will differ from the book's; the *shape* should not.

## Course Specification

---

This page is the course file. It states what HEC requires of a course called *Analysis of Algorithms* and shows where each requirement is met. Three editions are in force in different programmes, so all three are traced.

### Course particulars

---

| Item                | Value                                                                                                                                                                                                                                                                                |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Course title        | Analysis of Algorithms (HEC 2023). HEC 2017 and HEC 2025 both call it <i>Design and Analysis of Algorithms</i>                                                                                                                                                                       |
| Programme           | BS Information Technology. Under HEC 2023 this is a <b>computing core</b> course — compulsory in all ten BS computing degrees — in Semester 7. Under HEC 2017 it was a BSCS core course and <i>absent from BSIT entirely</i> ; under HEC 2025 it is a BSCS major core in Semester IV |
| Credit hours        | 3 (3–0): three credit hours of lectures, no separate laboratory credit. The laboratory work in this handout is therefore done in lecture time and as homework                                                                                                                        |
| Prerequisite        | <i>Data Structures</i> (HEC 2023). HEC 2017 required <i>Data Structures and Algorithms</i> ; HEC 2025 publishes no prerequisites at all                                                                                                                                              |
| Duration            | Sixteen teaching weeks, midterm in Week 8, project due Week 16 (Appendix G)                                                                                                                                                                                                          |
| Course introduction | The HEC 2023 statement, verbatim: “Detailed study of the basic notions of the design of algorithms and the underlying data structures. Several measures of complexity are introduced. Emphasis on the structure, complexity, and efficiency of algorithms.”                          |
| Set texts           | As published: Cormen, Leiserson, Rivest and Stein, <i>Introduction to Algorithms</i> ; Kleinberg and Tardos, <i>Algorithm Design</i> ; Sedgewick and Wayne, <i>Algorithms</i> . See the note below on the edition, and Appendix E                                                    |

#### Four defects in the published documents

Recorded here so that a reader comparing this handout with the HEC booklet is not confused, and so that the department’s own course file does not inherit them.

**1. The 2023 CLO labels are mis-aligned with their own text.** In the 2023 booklet CLO-1 sits against the first half of outcome 1 and CLO-2 against its continuation, leaving 3, 5 and 7 dangling. The 2017 listing of the same eight outcomes is correctly numbered, and *that* is the numbering used below.

**2. The Bloom’s Taxonomy column is empty in both editions**, for all eight outcomes — although the neighbouring Data Structures table fills it in. The levels in the table below are therefore *assigned by this department*, and are marked as such.

**3. “Corman”** is the booklets’ misspelling of **Cormen**, in both 2017 and 2023.

**4. The 3rd edition (2009) is cited; the 4th edition (2022) is current.** Appendix E cites the 4th and keeps the 3rd beside it, because chapter numbers moved between them.

### Coverage of the HEC 2023 course outline

The published outline is a single run-on paragraph. Each of its clauses is traced below, in the order HEC writes them. Nothing is omitted.

| HEC 2023 clause                                                                          | Chapter    | Where exactly                                                                                                         |
|------------------------------------------------------------------------------------------|------------|-----------------------------------------------------------------------------------------------------------------------|
| “Introduction; role of algorithms in computing”                                          | 1          | What an algorithm is, why the question is asymptotic, and the model of computation the whole book assumes             |
| “Analysis on nature of input and size of input”                                          | 2          | What “size” means for an array, a graph, a matrix and a number; best, expected and worst case                         |
| “Asymptotic notations; Big-O, Big $\Omega$ , Big $\Theta$ , little-o, little- $\omega$ ” | 3          | All five defined formally and proved from the definitions                                                             |
| “Sorting Algorithm analysis”                                                             | 8, 9, 10   | Merge sort, quicksort, and the lower bound that bounds them both                                                      |
| “loop invariants”                                                                        | 4          | Initialisation, maintenance, termination; insertion sort proved correct                                               |
| “Recursion and recurrence relations”                                                     | 5, 6       | Substitution, recursion trees, and the master theorem                                                                 |
| “Algorithm Design Techniques”                                                            | 7–9, 14–16 | The four the outline names, each with the property that makes it applicable                                           |
| “Brute Force Approach”                                                                   | 7          | Taken seriously: the baseline every other technique must beat, and the cases where it wins                            |
| “Divide-and-conquer approach; Merge, Quick Sort”                                         | 8, 9       | Both, in full, with the average-case analysis of quicksort                                                            |
| “Greedy approach”                                                                        | 14         | Greedy-choice and optimal substructure, proved by exchange argument                                                   |
| “Dynamic programming; Elements of Dynamic Programming”                                   | 15, 16     | The two elements as their own chapter, then the classic problems                                                      |
| “Search trees”                                                                           | 12         | Revisited for analysis: why the height is what it is, and what that costs                                             |
| “Heaps”                                                                                  | 12         | Likewise, with the priority-queue bounds Chapters 19 and 20 depend on                                                 |
| “Hashing”                                                                                | 13         | Expected versus worst case, load factor, and what “constant time” actually promises                                   |
| “Graph algorithms, shortest paths, sparse graphs”                                        | 18, 20     | Representation and sparsity first, because sparsity changes every bound in the part; then single-source and all-pairs |

| HEC 2023 clause                      | Chapter | Where exactly                                                     |
|--------------------------------------|---------|-------------------------------------------------------------------|
| “String matching”                    | 21      | Naive, Rabin–Karp and Knuth–Morris–Pratt                          |
| “Introduction to complexity classes” | 22      | P, NP, NP-hard and NP-complete, with one reduction worked in full |

### Course learning outcomes

The eight outcomes HEC publishes, in the 2017 numbering (see defect 1 above). Bloom levels are assigned by this department, because the published tables leave the column blank.

| CLO   | By the end of the course the student will be able to                                                                                         | Bloom         | Chapters                                     |
|-------|----------------------------------------------------------------------------------------------------------------------------------------------|---------------|----------------------------------------------|
| CLO-1 | Explain what is meant by “best”, “expected” and “worst” case behaviour of an algorithm                                                       | C2 Understand | 2, 9                                         |
| CLO-2 | Identify the characteristics of data, or other conditions or assumptions, that lead to different behaviours                                  | C4 Analyse    | 2, 9, 13, 18                                 |
| CLO-3 | Determine informally the time and space complexity of simple algorithms                                                                      | C3 Apply      | 1, 3, 17, and every chapter’s worked example |
| CLO-4 | List and contrast standard complexity classes                                                                                                | C2 Understand | 22                                           |
| CLO-5 | Use big $O$ , $\Omega$ and $\Theta$ notation formally to give asymptotic bounds on time and space complexity                                 | C3 Apply      | 3, 5, 6                                      |
| CLO-6 | Use the strategies — brute force, greedy, divide-and-conquer and dynamic programming — to solve an appropriate problem                       | C3 Apply      | 7, 8, 9, 14, 15, 16                          |
| CLO-7 | Solve problems using graph algorithms, including single-source and all-pairs shortest paths and at least one minimum spanning tree algorithm | C3 Apply      | 18, 19, 20                                   |
| CLO-8 | Trace and/or implement a string-matching algorithm                                                                                           | C3 Apply      | 21                                           |

#### 💡 One chapter the outline does not name, and why it is here

**Chapter 19, Minimum Spanning Trees**, appears nowhere in the outline paragraph — and **CLO-7 requires it by name**: “at least one minimum spanning tree algorithm”. The outline and the outcomes disagree, and this handout follows the outcome, because that is what a student is assessed against.

Two further chapters go beyond the letter of the outline and are justified rather than smuggled in. **Chapter 11 (Selection)** is the natural companion to Chapter 10’s lower bound and costs one lecture. **Chapter 17 (Amortised Analysis)** is needed before Chapter 13’s hash-table claim or Chapter 19’s union-find can be stated honestly; without it, “constant

time” in those chapters would have to be asserted rather than proved, and CLO-3 asks for complexity to be *determined*.

### Coverage of the HEC 2025 outcomes

HEC 2025 publishes no course contents for any course — only outcomes. Its five bullets for this course map onto the same chapters, so that a course file written against either edition has something to check.

| HEC 2025 outcome                                                               | Chapters          | Met by                                                                                                    |
|--------------------------------------------------------------------------------|-------------------|-----------------------------------------------------------------------------------------------------------|
| Design efficient algorithms for common computational problems                  | 7–9, 14–16, 18–21 | CLO-6 and CLO-7                                                                                           |
| Analyze algorithm complexity using Big O notation                              | 3, 5, 6           | CLO-5                                                                                                     |
| Solve problems involving recursion, divide-and-conquer and dynamic programming | 5–9, 15, 16       | CLO-6                                                                                                     |
| Compare different algorithmic approaches to problem-solving                    | 7, 14–16, 22      | CLO-6; the comparison tables that close Parts II and IV                                                   |
| Demonstrate correctness and optimality of algorithms                           | 4, 10, 14, 20     | Loop invariants in 4; the lower bound in 10; exchange arguments in 14; Dijkstra’s correctness proof in 20 |

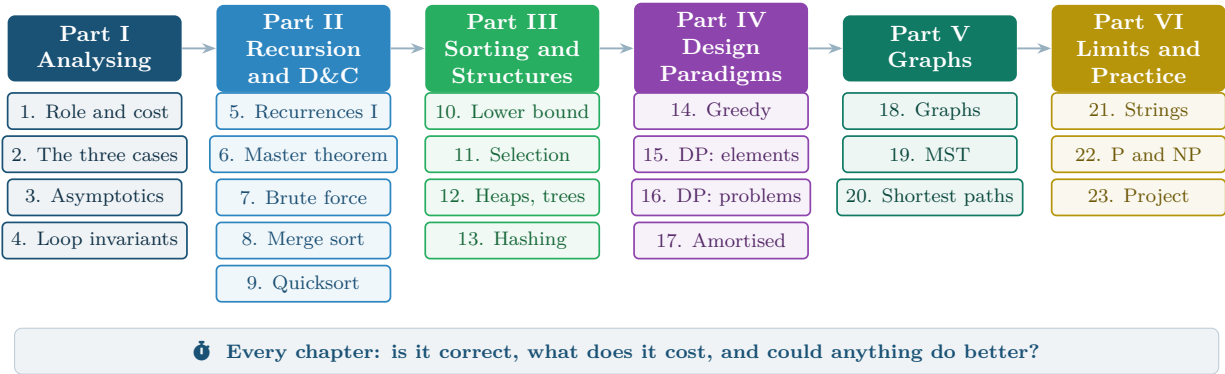
#### Where this course sits in the degree, and the one awkwardness

HEC 2023 places this course in **Semester 7** and *Artificial Intelligence* in Semester 3 — so a student meets heuristic search four semesters before meeting the analysis that would let them reason about its cost. HEC 2025 restores the sensible order, putting algorithms in Semester IV and AI in Semester V.

If your department follows the 2023 scheme, this handout assumes nothing from the later courses and can be taught as published. If it follows 2025, this course comes earlier and Chapters 12 and 13 may need more time, because Data Structures will be fresher but less practised.

# The Course at a Glance

---



**Figure F.1:** The six parts and the chapters in each. Chapter 10 is the hinge: it is the first result in the book about a *problem* rather than about an algorithm.

*This course is the published continuation of Data Structures and sits beside Theory of Automata. Appendix F states what belongs to each.*

## Notation

| Symbol                               | Meaning                                                                                                                                              |
|--------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| $n$                                  | The size of the input. Chapter 2 is about what that sentence hides                                                                                   |
| $T(n)$                               | The running time on an input of size $n$ , in the model of Chapter 1                                                                                 |
| $\lg n$                              | $\log_2 n$ . Used throughout; $\ln$ is natural and $\log$ without a base means the base does not matter                                              |
| $O(f(n))$                            | An asymptotic <i>upper</i> bound: $T$ grows no faster than $f$                                                                                       |
| $\Omega(f(n))$                       | An asymptotic <i>lower</i> bound                                                                                                                     |
| $\Theta(f(n))$                       | Both: $T$ grows exactly as fast as $f$                                                                                                               |
| $o(f(n)), \omega(f(n))$              | Strict versions of $O$ and $\Omega$ — strictly slower, strictly faster                                                                               |
| $A[1..n]$                            | An array of $n$ elements, indexed <b>from 1</b> , as the pseudocode does. The C++ in <code>code/</code> indexes from 0, and says so where it matters |
| $ V ,  E $                           | The number of vertices and edges of a graph. Chapter 18 explains why bounds are written in both                                                      |
| $\hat{c}_i$                          | The amortised cost of the $i$ th operation (Chapter 17)                                                                                              |
| $\Phi$                               | A potential function (Chapter 17)                                                                                                                    |
| P, NP                                | The complexity classes of Chapter 22                                                                                                                 |
| $\leq_p$                             | Polynomial-time reducibility (Chapter 22)                                                                                                            |
| $\lceil x \rceil, \lfloor x \rfloor$ | Ceiling and floor                                                                                                                                    |

### ⚠ Three notational habits worth breaking now

**$O$  is not  $\Theta$ .** “Merge sort is  $O(n^2)$ ” is *true* and nearly useless. If you mean the bound is tight, write  $\Theta$ . Chapter 3 is blunt about this because the examination is.

**= in “ $T(n) = O(n^2)$ ” is not equality.** It is set membership by long-standing abuse:  $O(n^2)$  is a *set of functions* and  $T$  belongs to it. You may never write  $O(n^2) = T(n)$ , and  $O(n) = O(n^2)$  is meaningless.

**Constants are discarded, not absent.**  $\Theta(n)$  with a constant of  $10^6$  loses to  $\Theta(n^2)$  with a constant of 1 until  $n$  exceeds a million. The measured strand in every chapter exists to keep this in view.

## Prerequisite Self-Check

---

Twelve questions. If you can answer ten, begin at Chapter 1. If you cannot, Appendix B covers the mathematics and your Data Structures notes cover the rest — and the gap is worth closing in Week 1 rather than Week 9.

1. Write the sum  $1 + 2 + \dots + n$  in closed form, and say why. (Appendix B)
2. Evaluate  $\sum_{i=0}^k 2^i$ . What is it approximately, for large  $k$ ? (Appendix B)
3. Simplify  $\log_2(n^3)$  and  $2^{\log_2 n}$ . (Appendix B)
4. State the principle of mathematical induction, and use it to show  $\sum_{i=1}^n i = n(n+1)/2$ . (Appendix B)
5. A binary tree of height  $h$  has at most how many leaves? Why? (Data Structures)
6. What is the height of a binary heap containing  $n$  elements? (Chapter 12)
7. Give the worst-case height of an unbalanced binary search tree on  $n$  keys, and the input that produces it. (Chapter 12)
8. Describe how a hash table resolves collisions by chaining, and what happens as the load factor rises. (Chapter 13)
9. What is the difference between an adjacency matrix and an adjacency list, in space? (Chapter 18)
10. If a fair coin is tossed  $n$  times, what is the expected number of heads? (Appendix B)
11. In C++, what is the difference between `std::vector` and a raw array, and which one has  $O(1)$  indexing? (Appendix A)
12. Write a function that reverses an array in place. What does it cost? (Programming Fundamentals)



Notice what is *not* on that list: calculus, linear algebra, and any particular cleverness. This course needs fluency with sums, logarithms and induction, comfort with the data structures you already built, and a willingness to finish a proof rather than stop when the idea is clear. It also needs you to be able to compile and run a C++ program. Appendix A sets that up in one sitting; do it before Week 2, because from Chapter 1 onward every chapter asks you to measure something.

# Contents

---

|                                                                       |             |
|-----------------------------------------------------------------------|-------------|
| <b>How to Use This Handout</b>                                        | <b>ii</b>   |
| <b>Course Specification</b>                                           | <b>iv</b>   |
| <b>The Course at a Glance</b>                                         | <b>viii</b> |
| <b>Notation</b>                                                       | <b>ix</b>   |
| <b>Prerequisite Self-Check</b>                                        | <b>x</b>    |
| <br>                                                                  |             |
| <b>I   Analysing an Algorithm</b>                                     | <b>1</b>    |
| <br>                                                                  |             |
| <b>1   The Role of Algorithms, and What Efficient Means</b>           | <b>3</b>    |
| 1.1   Two Programs That Agree on Every Answer . . . . .               | 3           |
| 1.2   The Model: What Counts as One Step . . . . .                    | 4           |
| 1.3   Why the Question Is Asymptotic . . . . .                        | 5           |
| 1.4   The Claim, Measured . . . . .                                   | 6           |
| 1.5   Space, and the Trade Against Time . . . . .                     | 7           |
| <br>                                                                  |             |
| <b>2   Input Size, Input Nature, and the Three Cases</b>              | <b>11</b>   |
| 2.1   The Size of the Input . . . . .                                 | 11          |
| 2.2   The Nature of the Input: Three Cases . . . . .                  | 13          |
| 2.3   Which Case Should You Report? . . . . .                         | 15          |
| <br>                                                                  |             |
| <b>3   Asymptotic Notation</b>                                        | <b>19</b>   |
| 3.1   Three Bounds, Defined . . . . .                                 | 19          |
| 3.2   Proving a Bound From the Definition . . . . .                   | 20          |
| 3.3   The Strict Bounds: $o$ and $\omega$ . . . . .                   | 21          |
| 3.4   Properties, and the Analogy With Inequality . . . . .           | 22          |
| 3.5   The Limit Test . . . . .                                        | 23          |
| 3.6   What $\Theta$ Asserts, Measured . . . . .                       | 24          |
| 3.7   Four Abuses, and What Each Conceals . . . . .                   | 26          |
| <br>                                                                  |             |
| <b>4   Correctness: Loop Invariants</b>                               | <b>30</b>   |
| 4.1   Why Testing Is Not Enough . . . . .                             | 30          |
| 4.2   The Loop Invariant . . . . .                                    | 31          |
| 4.3   Insertion Sort, Proved . . . . .                                | 32          |
| 4.4   Binary Search, Where the Invariant Is About Exclusion . . . . . | 33          |
| 4.5   Termination: The Obligation People Skip . . . . .               | 34          |
| 4.6   A Loop Whose Invariant Holds and Which Never Finishes . . . . . | 35          |

|     |                                                        |    |
|-----|--------------------------------------------------------|----|
| II  | Recursion and Divide-and-Conquer                       | 39 |
| 5   | Recurrences I: Substitution and Recursion Trees        | 41 |
| 6   | Recurrences II: The Master Theorem                     | 42 |
| 7   | Brute Force, and When It Is the Right Answer           | 43 |
| 8   | Divide and Conquer: Merge Sort and the Pattern         | 44 |
| 9   | Quicksort, Randomisation and the Average Case          | 45 |
| III | Sorting, Selection and the Structures Beneath          | 46 |
| 10  | The Comparison Lower Bound, and Sorting in Linear Time | 48 |
| 11  | Selection and Order Statistics                         | 49 |
| 12  | Heaps and Search Trees, Revisited for Analysis         | 50 |
| 13  | Hashing, and What Constant Time Actually Promises      | 51 |
| IV  | The Design Paradigms                                   | 52 |
| 14  | Greedy Algorithms                                      | 54 |
| 15  | Dynamic Programming I: The Elements                    | 55 |
| 16  | Dynamic Programming II: Classic Problems               | 56 |
| 17  | Amortised Analysis                                     | 57 |
| V   | Graph Algorithms                                       | 58 |
| 18  | Graphs: Representation, Traversal and Sparsity         | 60 |
| 19  | Minimum Spanning Trees                                 | 61 |
| 20  | Shortest Paths: Single-Source and All-Pairs            | 62 |
| VI  | Strings, Limits and Practice                           | 63 |
| 21  | String Matching                                        | 65 |
| 22  | Complexity Classes: P, NP and NP-Completeness          | 66 |
| 23  | The Semester Project                                   | 67 |

|                                                     |           |
|-----------------------------------------------------|-----------|
| <b>Appendices</b>                                   | <b>68</b> |
| <b>A The C++ Toolkit, and How to Time Honestly</b>  | <b>69</b> |
| <b>B The Mathematics This Course Assumes</b>        | <b>70</b> |
| <b>C Answers to the Checkpoints</b>                 | <b>71</b> |
| <b>D Glossary</b>                                   | <b>72</b> |
| <b>E Resources and Further Reading</b>              | <b>73</b> |
| <b>F The Boundary with the Neighbouring Courses</b> | <b>74</b> |
| <b>G The Sixteen-Week Schedule</b>                  | <b>75</b> |
| G.1 The Schedule . . . . .                          | 75        |
| G.2 How the Weeks Map to the HEC Outline . . . . .  | 76        |

## List of Figures

---

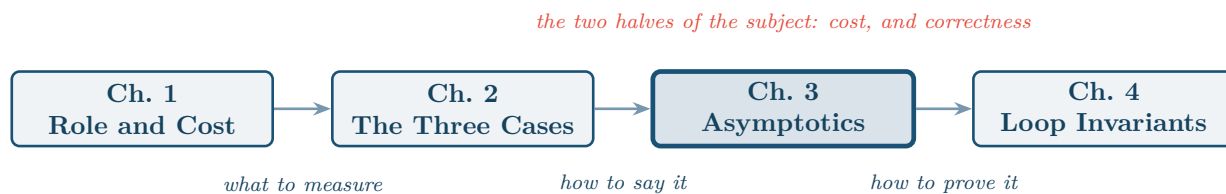
|      |                                                                                                                                                                                                                                                                                               |      |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|
| F.1  | The six parts and the chapters in each. Chapter 10 is the hinge: it is the first result in the book about a <i>problem</i> rather than about an algorithm. . . . .                                                                                                                            | viii |
| P1.1 | Reading path through Part I. Chapters 1–3 build the language of cost; Chapter 4 is the other half of the subject, and the one students most often skip. . . . .                                                                                                                               | 2    |
| 1.1  | The six orders of growth this book is mostly about, on a logarithmic vertical axis — note that each gridline is a factor of a hundred. At $n = 100$ the gap between $n \lg n$ and $n^2$ is already a factor of fifteen, and the gap between $n^2$ and $2^n$ does not fit on the page. . . . . | 5    |
| 4.1  | INSERTION-SORT. The outer loop’s invariant concerns the prefix $A[1..j-1]$ ; the inner loop’s concerns where <i>key</i> has got to. Lines are numbered so the proof can cite them. . . . .                                                                                                    | 32   |
| 4.2  | BINARY-SEARCH on a sorted array. The +1 on line 7 and the −1 on line 9 are what make the search range strictly smaller every iteration — Section 4.5 is about what happens when one of them is missing. . . . .                                                                               | 33   |
| 4.3  | EUCLID-GCD. Here the invariant is an equality — $\gcd(a, b)$ never changes — and the variant is simply $b$ , whose decrease is what makes the loop finite. . . . .                                                                                                                            | 34   |
| P2.1 | Reading path through Part II. Chapter 6 is the tool you will reach for most; Chapter 7 is the baseline that makes the rest worth having. . . . .                                                                                                                                              | 40   |
| P3.1 | Reading path through Part III. Chapter 10 is the hinge of the whole book; Chapters 12 and 13 revisit Data Structures for their analysis, not their implementation. . . .                                                                                                                      | 47   |
| P4.1 | Reading path through Part IV. Chapter 15 is the one to understand rather than memorise: the two properties in it decide whether greedy or dynamic programming applies at all. . . . .                                                                                                         | 53   |
| P5.1 | Reading path through Part V. Chapter 18 comes first because the representation decides the bounds: the same algorithm is a different cost on a sparse graph and a dense one. . . . .                                                                                                          | 59   |
| P6.1 | Reading path through Part VI. Chapter 22 is the only chapter whose central claim is that nobody knows; Chapter 23 is where you use the rest. . . . .                                                                                                                                          | 64   |

PART I

# Analysing an Algorithm

---

*Three questions are asked of every algorithm in this book, and Part I is the vocabulary for asking them: what does it cost, how does the input shape that cost, and how would you prove the thing is correct rather than merely believe it?*



**Figure P1.1:** Reading path through Part I. Chapters 1–3 build the language of cost; Chapter 4 is the other half of the subject, and the one students most often skip.

## The Role of Algorithms, and What Efficient Means

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part I — Analysing an Algorithm

### 🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **State** what an algorithm is, and **distinguish** it from a program that implements it.
- **Describe** the random-access machine model this course measures against, and **name** three things it deliberately ignores.
- **Explain** why the interesting question about an algorithm is asymptotic, and **compute** how long a given algorithm takes at a given input size.
- **Predict** the ratio by which a running time grows when the input doubles, from the algorithm's order of growth, and **check** the prediction by measurement.
- **Explain** why a faster computer running a worse algorithm loses, and **find** the input size at which it starts losing.

### 🔗 Before You Start

Nothing in this book. You need to be able to read a loop and to have written programs that sort or search — *Data Structures* is the published prerequisite and this chapter assumes only its first fortnight. Appendix A sets up the compiler, and you will want it from Section 1.4 onward.

### 📖 Key Terms in This Chapter

algorithm • problem • instance • correctness • random-access machine • RAM model • primitive operation • running time • input size • order of growth • asymptotic analysis • constant factor • space complexity • time-space trade-off

## 1.1 Two Programs That Agree on Every Answer

Here are two ways to decide whether a sorted array of a million names contains one particular name.

The first walks the array from the start, comparing as it goes, and stops when it finds the name or runs off the end. The second looks at the middle element, discards the half that cannot contain the name, and repeats.

Both are correct. Both return the same answer for every input. Run them on the same machine with the same compiler and the first takes, on a million names, about half a million comparisons; the second takes twenty. Not twenty thousand — twenty.

That gap is the subject of this course, and the first thing to notice about it is that it has nothing to do with programming. It is not a faster loop, a better compiler or a tighter data structure. It is a different *idea*, and the difference between the two ideas survives every change of language, machine and decade.

### Problem, Instance, Algorithm

A **problem** is a specification: a description of the input it accepts and the output it must produce. “Sorting” is a problem — given a sequence of  $n$  numbers, produce a permutation of them in non-decreasing order.

An **instance** is one particular input:  $\langle 31, 41, 59, 26 \rangle$ .

An **algorithm** is a finite, unambiguous procedure that produces the correct output for *every* instance. It is **correct** if it halts on every instance with the right answer — and “every” is doing real work in that sentence, which is why Chapter 4 exists.

A *program* is an algorithm written in a language, for a machine. This book analyses algorithms and measures programs, and is careful about which it is doing.

## 1.2 The Model: What Counts as One Step

To say an algorithm takes “half a million comparisons” we need to agree what a step is. Real machines make that awkward: an addition and a division cost differently, memory access costs differently depending on the cache, and the processor may be executing four instructions at once. So we measure against an idealisation.

### The Random-Access Machine

The **RAM model** assumes:

- instructions execute one at a time, with no concurrency;
- each **primitive operation** — arithmetic, comparison, assignment, reading or writing one memory cell, following one array index — costs one unit of time;
- memory is unlimited and every cell costs the same to reach.

The **running time**  $T(n)$  of an algorithm is then the number of primitive operations it performs on an input of size  $n$ .

### Lightbulb Three Lies in That Model, and Why They Are Worth Telling

Every clause of the definition is false of a real machine, deliberately.

**Not all operations cost the same.** On a current processor an integer addition takes about one cycle and an integer division about twenty. The model prices them equally.

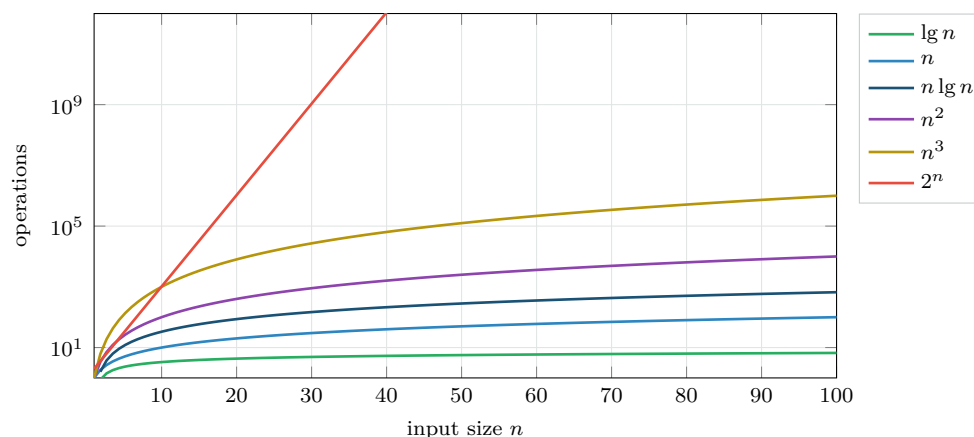
**Not all memory costs the same.** A read served by the level-1 cache takes about four cycles; one that goes to main memory takes a few hundred — a factor of fifty, which the model ignores entirely. Chapter 18 is where this starts to be visible in the measurements.

**Instructions do not execute one at a time.** Processors reorder, pipeline and predict branches.

The model is nevertheless the right one, for the reason the next section gives: every one of those lies is a *constant factor*, and the differences this course is about are not. A fifty-fold cache penalty is swamped by the difference between  $n$  and  $n^2$  before  $n$  reaches a thousand. Where the constant factors do matter — and they sometimes do — this book measures rather than models, which is what the *Try it yourself* boxes are for.

### 1.3 Why the Question Is Asymptotic

We do not ask how long an algorithm takes. We ask how the time *grows* with the input, and we answer with the dominant term, discarding constants. That sounds like giving up information. It is the opposite: it is the only way to say something that stays true.



**Figure 1.1:** The six orders of growth this book is mostly about, on a logarithmic vertical axis — note that each gridline is a factor of a hundred. At  $n = 100$  the gap between  $n \lg n$  and  $n^2$  is already a factor of fifteen, and the gap between  $n^2$  and  $2^n$  does not fit on the page.

## ⚠ What an exponential actually means

$2^n$  is not “slow”. It is a wall.

At  $n = 50$ , an algorithm doing  $2^n$  operations on a machine executing  $10^{10}$  of them a second takes

$$\frac{2^{50}}{10^{10}} = \frac{1.13 \times 10^{15}}{10^{10}} \approx 1.1 \times 10^5 \text{ s} \approx 31 \text{ hours.}$$

At  $n = 60$  it takes about **36 days**; at  $n = 80$ , about a hundred thousand years. Buying a computer a thousand times faster moves  $n = 50$  to  $n \approx 60$  and no further.

That is why Chapter 22 matters, and why “we will just use a faster machine” is not an answer to it.

### Worked Example: The Faster Computer Loses

Computer A executes  $10^{10}$  primitive operations a second and runs insertion sort, which takes about  $2n^2$  operations. Computer B executes  $10^7$  a second — a thousand times slower — and runs merge sort, which takes about  $50n \lg n$ . Sort ten million numbers on each.

### Step 1 — Computer A.

$$\frac{2 \times (10^7)^2}{10^{10}} = \frac{2 \times 10^{14}}{10^{10}} = 2 \times 10^4 \text{ s} = \mathbf{5 \text{ hours } 33 \text{ minutes}}$$

**Step 2 — Computer B.** First  $\lg(10^7) = 7\lg 10 = 7 \times 3.322 = 23.25$ .

$$\frac{50 \times 10^7 \times 23.25}{10^7} = 50 \times 23.25 = 1,163 \text{ s} = \mathbf{19} \text{ minutes } \mathbf{23} \text{ seconds}$$

**Step 3 — the comparison.** The machine that is *a thousand times slower* finishes **17 times sooner**, because its algorithm is better. The hardware advantage is not small and it is not enough.

**Step 4 — where does B overtake A?** Set the two times equal:

$$\frac{2n^2}{10^{10}} = \frac{50n \lg n}{10^7} \implies 2n = 5 \times 10^4 \lg n \implies n = 2.5 \times 10^4 \lg n.$$

There is no closed form, so iterate from  $n = 10^5$ :

$$2.5 \times 10^4 \times 16.6 = 4.15 \times 10^5 \rightarrow 4.66 \times 10^5 \rightarrow 4.71 \times 10^5 \rightarrow 4.71 \times 10^5.$$

So A wins below about  $n = 471,000$  and loses above it — and loses by more, the larger the input gets.

**Step 5 — what the arithmetic is really saying.** Not that constant factors are unimportant. A constant of 25 between the two algorithms bought A nearly half a million elements' worth of advantage, which for many real problems is the whole range of interest. It is saying that constants buy a *bounded* advantage and a better order of growth buys an unbounded one, and that you should know which regime you are in.

## 1.4 The Claim, Measured

The worked example is arithmetic about a hypothetical machine. The claim it rests on — that a quadratic algorithm's time quadruples when the input doubles, and an  $n \lg n$  one's roughly doubles — is testable on the machine in front of you, and ought to be tested.

### 💡 What the ratios should be

For  $T(n) = cn^2$ ,

$$\frac{T(2n)}{T(n)} = \frac{c(2n)^2}{cn^2} = 4,$$

independent of  $c$  and of  $n$ . The constant cancels, which is exactly why the ratio is worth measuring: it tests the *order of growth* without needing to know anything about the machine.

For  $T(n) = cn \lg n$ ,

$$\frac{T(2n)}{T(n)} = \frac{2n(\lg n + 1)}{n \lg n} = 2 + \frac{2}{\lg n},$$

which is a little above 2 and drifts *down* towards 2 as  $n$  grows. At  $n = 1000$  it predicts  $2 + 2/9.97 = 2.20$ .

### 📊 Worked Example: The Prediction Against the Measurement

Running `code/ch01_growth.cpp` on the machine described in Appendix A (MSVC 19.44, /O2), median of five runs, random permutations:

| $n$    | insertion (ms) | ratio | merge (ms) | ratio |
|--------|----------------|-------|------------|-------|
| 1,000  | 0.15           | —     | 0.06       | —     |
| 2,000  | 0.58           | 3.87  | 0.14       | 2.22  |
| 4,000  | 2.41           | 4.13  | 0.30       | 2.20  |
| 8,000  | 9.35           | 3.88  | 0.65       | 2.14  |
| 16,000 | 37.08          | 3.97  | 1.51       | 2.32  |
| 32,000 | 145.86         | 3.93  | 2.94       | 1.96  |

**Insertion sort.** Predicted 4; measured 3.87, 4.13, 3.88, 3.97, 3.93. Mean 3.96.

**Merge sort.** Predicted  $2 + 2/\lg n$  — 2.20 at  $n = 1000$ , falling to 2.14 by  $n = 8000$ . Measured 2.22, 2.20, 2.14, 2.32, 1.96. The first three agree to within a per cent; the last two straddle the prediction.

**Read the last column honestly.** The 2.32 and the 1.96 are not the theory failing. They are two measurements of a quantity near 2.15 with a few per cent of noise, and a ratio magnifies noise in both terms. The way to tell noise from a real effect is to look at the *pattern*: insertion’s five ratios scatter about 4 and merge’s about 2.2, and neither drifts.

**And notice what the measurement cost.** At  $n = 32,000$  insertion sort took 146 ms and merge sort 2.9 ms — a factor of 50. At  $n = 1,000$  the factor was 2.5. The gap is not fixed; it widens with every doubling, and that is the whole content of “asymptotically faster”.

### ⚠ The asymptotics are not the whole story, and this book says so twice

The same program, run on small inputs, gives this:

| $n$ | insertion ( $\mu\text{s}$ ) | merge ( $\mu\text{s}$ ) |                |
|-----|-----------------------------|-------------------------|----------------|
| 20  | 0.20                        | 0.30                    | insertion wins |
| 40  | 0.50                        | 0.60                    | insertion wins |
| 60  | 0.70                        | 1.00                    | insertion wins |
| 70  | 1.20                        | 1.10                    | merge wins     |
| 100 | 2.20                        | 1.60                    | merge wins     |
| 200 | 7.40                        | 4.60                    | merge wins     |

The crossover is at about  $n = 65$  on this machine. Below it the “worse” algorithm is faster, every time, because merge sort’s recursion and its auxiliary array cost more than insertion sort’s whole quadratic loop on sixty elements.

This is not a curiosity. It is why every production sort in the world — the C++ standard library’s included — switches to insertion sort below a threshold of roughly this size. Chapter 9 returns to it.

## 1.5 Space, and the Trade Against Time

Time is not the only resource. **Space complexity** counts the memory an algorithm needs beyond its input, and the two are often tradeable: a lookup table turns a computation into a memory access, and Chapter 15’s whole technique is spending memory to avoid recomputation.

### 💡 Which Resource Binds?

For most of the twentieth century, memory was the scarce one. It is now usually not: a machine with 32 GB can hold an array of four billion integers, and most problems that fit in a lecture do not come close.

What has replaced it is *memory locality*. An algorithm using twice the memory but touching it in order can comfortably beat one using half as much in a scattered pattern, because of the cache the RAM model promised to ignore. Chapter 18 measures exactly this.

So: analyse space because the exam asks and because occasionally it binds; but when two algorithms have the same order of growth and different speeds, look at the access pattern before the byte count.

**! Common Pitfalls**

- **Confusing the algorithm with the program.** “My sort is slow” is usually a statement about an implementation; “insertion sort is quadratic” is a statement about an idea. Fixing the first will not change the second.
- **Timing an unoptimised build.** A debug build can be five to ten times slower and — worse — slower by *different* factors for different code, so the comparison is meaningless. Appendix A insists on /O2.
- **Timing once.** A single run contains whatever else the machine was doing. Every measurement in this book is the median of several.
- **Believing the asymptotics at small  $n$ .** The crossover above is at  $n = 65$ . If your inputs are always smaller than that, the better order of growth is buying you nothing.
- **Believing the constants matter at large  $n$ .** The symmetric error. At  $n = 32,000$  the “unimportant” factor of 50 was entirely due to the order of growth.
- **Reporting a mean of timings.** One scheduler interruption drags a mean a long way. Use the median, and say how many runs.

**🔧 Try It Yourself: Growth, Measured**

Reproduces both tables above on your own machine. Appendix A sets up the compiler. Expect twenty minutes, most of it reading the output.

```
// ch01_growth.cpp -- what "the question is asymptotic" looks like.
// Build: cl /O2 /EHsc /std:c++17 ch01_growth.cpp      (see Appendix A)

// --- 1. The two algorithms, written plainly -----
static void insertion_sort(std::vector<int>& a) {
    for (size_t j = 1; j < a.size(); ++j) {
        int key = a[j];
        size_t i = j;
        while (i > 0 && a[i - 1] > key) { a[i] = a[i - 1]; --i; }
        a[i] = key;
    }
}

static void merge_sort(std::vector<int>& a, std::vector<int>& buf,
                      size_t lo, size_t hi) {
    if (hi - lo < 2) return;
    size_t mid = lo + (hi - lo) / 2;
    merge_sort(a, buf, lo, mid);
    merge_sort(a, buf, mid, hi);
    size_t i = lo, j = mid, k = lo;
    while (i < mid && j < hi) buf[k++] = (a[i] <= a[j]) ? a[i++] : a[j++];
    while (i < mid) buf[k++] = a[i++];
    while (j < hi)  buf[k++] = a[j++];
    std::copy(buf.begin() + lo, buf.begin() + hi, a.begin() + lo);
}

// --- 2. Timing that is honest -----
// The MEDIAN of several runs: a mean is dragged by one scheduler
// hiccup, and the minimum flatters the cache.
template <class F>
static double time_ms(F&& f, int reps) {
    std::vector<double> t;
    for (int r = 0; r < reps; ++r) {
        auto t0 = std::chrono::steady_clock::now();
        f();
        auto t1 = std::chrono::steady_clock::now();
        t.push_back(
```

```

        std::chrono::duration<double, std::milli>(t1 - t0).count());
    }
    std::sort(t.begin(), t.end());
    return t[t.size() / 2];
}

// --- 3. Double n each time, and watch the ratios -----
// Predicted: 4 for insertion sort, 2 + 2/lg(n) for merge sort.
for (size_t n = 1000; n <= 32000; n *= 2) {
    auto base = random_input(n, 12345);
    std::vector<int> buf(n);
    double ti = time_ms([&]{ auto a = base; insertion_sort(a); }, 5);
    double tm = time_ms([&]{ auto a = base;
        merge_sort(a, buf, 0, n); }, 5);
    printf("%8zu %12.2f %7.2f %12.2f %7.2f\n",
        n, ti, ti / prev_i, tm, tm / prev_m);
    prev_i = ti; prev_m = tm;
}

// --- 4. Now find where they cross -----
// The other half of the point: below the crossover the "worse"
// algorithm is faster, and every production sort knows it.
for (size_t n = 10; n <= 200; n += 10) {
    /* same two timings, many more repetitions because the times
       are microseconds; print both and mark which one won */
}

```

**What to notice.** Your absolute times will differ from the book's — a different processor, a different compiler, a different afternoon. The *ratios* should not. If your insertion-sort column is not near 4, something is wrong with the measurement, not with the theory: check that you built with optimisation on, and that nothing else was running.

### ? Checkpoint — Test Yourself

1. An algorithm takes 3 seconds on an input of size 1,000 and 48 seconds on an input of size 4,000. What is its order of growth, and how did you decide?
2. Computer A runs  $n^3$  operations at  $10^9$  per second; computer B runs  $100n^2$  at  $10^7$  per second. At  $n = 10^4$ , which finishes first, and by what factor?
3. The RAM model prices a cache hit and a main-memory access the same, although they differ by a factor of about fifty. Give one situation in which that lie is harmless and one in which it is not.

### ✓ Chapter Summary

- A problem is a specification, an instance is one input, an algorithm is a procedure correct on *every* instance. A program implements an algorithm; this book analyses the second and measures the first.
- The RAM model costs every primitive operation at one unit. All three of its assumptions are false of real hardware, and all three are constant factors — which is why the model is still the right one.
- We ask how running time *grows*, not what it is, because the growth is the part that survives a change of machine.
- Constants buy a bounded advantage; a better order of growth buys an unbounded one. A computer 1,000 times faster still lost by a factor of 17, and the crossover was at  $n \approx 471,000$ .
- Doubling the input multiplies a quadratic time by 4 and an  $n \lg n$  time by  $2 + 2/\lg n$ . Measured on this machine: 3.96 and 2.17.

- Below  $n \approx 65$  insertion sort beat merge sort on the same machine. Asymptotics describe the limit, and the limit is not everywhere.
- Measure with optimisation on, several times, and report the median.

### Review Questions

1. **(MCQ)** The RAM model assumes: (a) memory access is free; (b) every primitive operation costs one unit of time; (c) instructions execute in parallel; (d) arithmetic is exact to arbitrary precision.
2. **(MCQ)** If  $T(n) = cn^2$ , then  $T(2n)/T(n)$  equals: (a) 2; (b)  $2 + 2/\lg n$ ; (c) 4; (d) it depends on  $c$ .
3. **(Short)** Explain why discarding constant factors makes an analysis *more* useful rather than less.
4. **(Short)** Give the three assumptions of the RAM model and, for each, one way real hardware violates it.
5. **(Short)** A colleague says “we do not need a better algorithm, we will buy a faster server”. Give the shortest argument that settles the question, and say what information you would need first.
6. **(Applied)** Algorithm P takes  $0.5n^2$  operations and algorithm Q takes  $200n \lg n$ . Find the input size at which Q overtakes P, showing the iteration. Then say what you would recommend to a team whose inputs never exceed 2,000.
7. **(Applied)** Design a measurement that would distinguish an  $O(n \lg n)$  algorithm from an  $O(n)$  one, given only a compiled binary you can time. State the sizes you would use, how many repetitions, what you would compute, and what result would make you say the evidence is inconclusive.

## Input Size, Input Nature, and the Three Cases

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part I — Analysing an Algorithm

### 🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Define** the size of an input for an array, a matrix, a graph and a number, and **justify** why a number's size is its bit count.
- **Explain** what is meant by the best, expected and worst case behaviour of an algorithm (CLO-1).
- **Identify** the characteristics of the data, or the other conditions and assumptions, that lead an algorithm to different behaviours (CLO-2).
- **Derive** the best, worst and average case running time of insertion sort and linear search, and **say** which to report and why.
- **Distinguish** an average taken over inputs from an expectation taken over an algorithm's own random choices.

### 🔗 Before You Start

Chapter 1 — the RAM model and the idea of an order of growth. You will also need to be comfortable that  $\lg$  means  $\log_2$ , and that  $\sqrt{2^b} = 2^{b/2}$ .

### 📖 Key Terms in This Chapter

input size • bit length • sparse • dense • best case • worst case • average case • expected running time • input distribution • uniform random permutation • randomised algorithm • adversary • pseudo-polynomial

## 2.1 The Size of the Input

Everything in Chapter 1 was written as a function of  $n$  without ever saying what  $n$  counts. For an array that is obvious enough. For the other things algorithms take as input it is a decision, and a decision that changes the answer.

### 📑 Input Size

The **input size** is whatever parameter the running time is most naturally expressed in — but it must be a measure of *how much input there is*, not of what the input says.

| Input               | Size, by convention                                                                                                      |
|---------------------|--------------------------------------------------------------------------------------------------------------------------|
| array, list         | $n$ , the number of elements                                                                                             |
| string              | $n$ , the number of characters                                                                                           |
| $n \times n$ matrix | $n$ , the side — so an $O(n^3)$ algorithm is <i>linear</i> in the $n^2$ numbers it reads, which is worth saying out loud |
| graph               | two parameters: $ V $ vertices and $ E $ edges                                                                           |

|               |                                                    |
|---------------|----------------------------------------------------|
| <b>number</b> | $b$ , the number of <i>bits</i> needed to write it |
|---------------|----------------------------------------------------|

Two of those rows need an argument.

### Graphs take two parameters, and that is not a technicality

A graph has  $|V|$  vertices and  $|E|$  edges, and  $|E|$  can be anywhere from 0 to  $|V|(|V| - 1)/2$ . A bound of  $O(|V| + |E|)$  and a bound of  $O(|V|^2)$  are the same thing on a **dense** graph and wildly different on a **sparse** one.

#### Worked Example: The Same Bound, Two Graphs

Breadth-first search is  $O(|V| + |E|)$  from an adjacency list.

**A road network:** 2 million junctions, and each junction meets about three roads, so  $|E| \approx 3 \times 10^6$ .

$$|V| + |E| \approx 2 \times 10^6 + 3 \times 10^6 = 5 \times 10^6.$$

**A social graph of the same vertex count,** each person connected to 500 others, so  $|E| \approx 2 \times 10^6 \times 500/2 = 5 \times 10^8$ .

$$|V| + |E| \approx 5 \times 10^8.$$

**A factor of 100,** on the same algorithm, at the same  $|V|$ . This is why Chapter 18 comes before the graph algorithms rather than after them, and why every bound in Part V is quoted in both parameters.

### A number's size is its length, not its value

This one students get wrong, and it is the root of a famous confusion that Chapter 16 and Chapter 22 both depend on.

#### Why not just use $N$ itself?

Because the input to a program is *written down*, and a number  $N$  is written in about  $b = \lfloor \lg N \rfloor + 1$  bits. Typing one more digit multiplies  $N$  by ten while adding three and a bit bits of input.

If we measured size by value, then an algorithm that loops  $N$  times would be “linear” — and we would be calling an algorithm efficient when doubling the *length* of its input multiplies its running time by  $2^b$ .

So: **size** =  $b \approx \lg N$ , and an algorithm that loops  $N$  times is  $O(2^b)$  — exponential. An algorithm whose cost is polynomial in  $N$  but exponential in  $b$  is called **pseudo-polynomial**; the 0/1 knapsack algorithm of Chapter 16 is the standard example, and Chapter 22 explains why that distinction decides whether a problem counts as solved.

#### Worked Example: The Claim, Measured

`code/ch02_cases.cpp` tests a prime by trial division — a loop running about  $\sqrt{N}/2$  times. Since  $\sqrt{N} = \sqrt{2^b} = 2^{b/2}$ , adding *two* bits should exactly double the work. Each row below is a prime two bits longer than the row above; median of three runs.

| bits | $N$                 | time (ms) | ratio |
|------|---------------------|-----------|-------|
| 32   | 4,294,967,291       | 0.22      | —     |
| 34   | 17,179,869,143      | 0.44      | 2.01  |
| 36   | 68,719,476,731      | 0.86      | 1.94  |
| 38   | 274,877,906,899     | 1.76      | 2.06  |
| 40   | 1,099,511,627,689   | 3.63      | 2.07  |
| 42   | 4,398,046,511,093   | 6.93      | 1.91  |
| 44   | 17,592,186,044,399  | 15.75     | 2.27  |
| 46   | 70,368,744,177,643  | 29.83     | 1.89  |
| 48   | 281,474,976,710,597 | 58.81     | 1.97  |

**Eight ratios, mean 2.02.** The prediction is exact and the measurement meets it.

**Now extrapolate, which is the point.** From 32 bits to 48 the time went from 0.22 ms to 58.81 ms — a factor of 267, against a predicted  $2^8 = 256$ . Carry that on:

- **64 bits:**  $2^8 \times 58.81 \text{ ms} \approx 15 \text{ seconds}$ .
- **128 bits:**  $2^{40} \times 58.81 \text{ ms} \approx 6.5 \times 10^{10} \text{ s} \approx \mathbf{2,000 \text{ years}}$ .
- **2048 bits**, which is what an RSA key is: there is no useful way to write the number.

**And notice what did *not* happen.** Going from 32 bits to 128 bits is going from a four-byte input to a sixteen-byte input — twelve extra bytes. The input grew by a factor of four and the running time by a factor of  $4 \times 10^{11}$ . That is what “exponential in the input size” means, and it is why measuring the size by the value would have hidden it completely.

## 2.2 The Nature of the Input: Three Cases

Fix the size. Two inputs of the same size can still cost an algorithm wildly different amounts, because an algorithm’s control flow depends on what the data *says*, not only on how much of it there is.

### Best, Worst and Average Case

Let  $T(x)$  be the number of primitive operations the algorithm performs on input  $x$ , and let  $X_n$  be the set of all inputs of size  $n$ .

| Case                                           | What it promises                                                                    |
|------------------------------------------------|-------------------------------------------------------------------------------------|
| <b>best:</b> $\min_{x \in X_n} T(x)$           | Nothing useful — a lower bound on an optimist’s hopes.                              |
| <b>worst:</b> $\max_{x \in X_n} T(x)$          | A <i>guarantee</i> : no input of size $n$ costs more than this.                     |
| <b>average:</b> $\mathbb{E}_{x \sim D} [T(x)]$ | What to expect <i>if</i> inputs really are drawn from $D$ — and $D$ must be stated. |

### ⚠ “Average case” is meaningless until you name a distribution

There is no such thing as a random input. There is only an input drawn from a distribution you have chosen, and the choice is a modelling assumption that can be wrong.

The conventional assumption for sorting is a **uniform random permutation**: every ordering of the  $n$  elements equally likely. It is a reasonable default and it is often false —

real data arrives partly sorted, or sorted by the wrong key, or in a few long runs. When you quote an average case, say what distribution it averages over. When you read one, check.

### Insertion sort, in all three cases

Insertion sort's inner loop shifts elements right until it finds the place for **key**. The number of shifts for element  $j$  is how many of the  $j$  elements before it are larger — the number of *inversions* involving  $j$ .

#### Worked Example: The Three Cases, Derived

**Best case — already sorted.** For every  $j$ ,  $a[j-1] \leq \text{key}$ , so the **while** test fails at once and does zero shifts. The outer loop still runs  $n-1$  times:

$$T(n) = c(n-1) = \Theta(n).$$

**Worst case — strictly decreasing.** Every element must travel to the front, so element  $j$  does  $j$  shifts:

$$\sum_{j=1}^{n-1} j = \frac{(n-1)n}{2} \approx \frac{n^2}{2} = \Theta(n^2).$$

**Average case — uniform random permutation.** For any pair  $(i, j)$  with  $i < j$ , the two orderings are equally likely, so the pair is inverted with probability  $\frac{1}{2}$ . There are  $\binom{n}{2}$  pairs, so by linearity of expectation the expected number of inversions is

$$\mathbb{E}[\text{inversions}] = \frac{1}{2} \binom{n}{2} = \frac{n(n-1)}{4} \approx \frac{n^2}{4} = \Theta(n^2).$$

**The conclusion, and it is the one that matters.** The average case is *exactly half* the worst case. A factor of two is a constant, and  $\Theta(\cdot)$  discards constants — so insertion sort's average and worst cases have the *same order of growth*, and the only case that differs is the best one.

#### Worked Example: One Algorithm, Three Orders of Growth, Measured

`code/ch02_cases.cpp` runs the same insertion sort on three shapes of input at each size; median of five runs, times in milliseconds.

| $n$    | sorted | ratio | random | ratio | reversed | ratio |
|--------|--------|-------|--------|-------|----------|-------|
| 2,000  | 0.002  | —     | 0.340  | —     | 0.634    | —     |
| 4,000  | 0.004  | 1.90  | 1.430  | 4.21  | 2.551    | 4.03  |
| 8,000  | 0.008  | 2.05  | 5.288  | 3.70  | 9.736    | 3.82  |
| 16,000 | 0.021  | 2.73  | 21.357 | 4.04  | 42.714   | 4.39  |
| 32,000 | 0.028  | 1.33  | 89.790 | 4.20  | 185.054  | 4.33  |

**The sorted column doubles** — 1.90, 2.05, 2.73, 1.33. The ratios are ragged because the absolute times are two to twenty-eight *microseconds*, where the clock's own resolution and one stray interrupt are a large fraction of the measurement. The honest reading is: these are five numbers consistent with linear growth and far too small to be consistent with quadratic, which is all the experiment is being asked to show.

**The random and reversed columns quadruple**, mean 4.04 and 4.14, as  $\Theta(n^2)$  requires.

**And now the prediction the derivation made.** Random should be half of reversed, because  $n^2/4$  is half of  $n^2/2$ . Measured:

0.54, 0.56, 0.54, 0.50, 0.49.

Five independent confirmations of a factor the theory produced from counting inversions. This is what it looks like when an analysis is right about more than the exponent.

**Finally, the spread.** At  $n = 32,000$ : 0.028 ms on sorted input, 185 ms on reversed. Same algorithm, same compiler, same machine, same input size — a factor of **6,600**. “How big is the input” does not come close to determining how long a program takes.

## 2.3 Which Case Should You Report?

💡 The convention, and the reasons for it

**Report the worst case unless you say otherwise.** Three reasons:

1. It is a *guarantee*. An average tells you about a population of runs; a worst case tells you about the run you are having now, which is the one with the deadline.
2. It is often what happens. For searching an absent key, or any algorithm that must examine everything to be sure, the worst case is the typical case.
3. It is frequently the same order as the average anyway — insertion sort above, where the factor between them is two.

**Report the best case almost never.** “This sort is  $O(n)$ ” with the best case quietly intended is the most common dishonest sentence in the subject.

**Report the average case when the worst case is both rare and pessimistic**, as with quicksort (Chapter 9) and hashing (Chapter 13) — and then state the distribution, because in both of those chapters there is an input that makes the worst case happen on purpose.

### 📅 Expected Running Time

A **randomised algorithm** makes random choices of its own. Its **expected running time** on an input  $x$  is the average over *those choices*, with  $x$  held fixed:

$$\mathbb{E}[T(x)], \quad \text{expectation over the algorithm's coin flips.}$$

### ⚠️ Two different averages, and the difference is the whole trick

**Average-case analysis** averages over inputs. You must assume a distribution on the input, and an adversary who knows your algorithm can choose an input outside it.

**Expected-time analysis** of a randomised algorithm averages over the algorithm's own coins, *for every input*. There is no assumption about the data at all. An adversary who knows your code still cannot pick a bad input, because he does not know your coins.

That is why Chapter 9 randomises: it converts an assumption about the world, which can be violated, into an assumption about a random number generator, which cannot. The bound goes from “ $\Theta(n \lg n)$  if your data is unsorted” to “ $\Theta(n \lg n)$  expected, on *every* input”.

### ❗ Common Pitfalls

- **Quoting one case and meaning another.** Always name it: “linear search is  $\Theta(n)$  in the worst case and  $\Theta(n)$  on average” is a real statement; “linear search is  $O(n)$ ” is a weaker one that people read as the worst case, correctly.

- **Measuring the size of a number by its value.** Then trial division looks linear, and the table in Section 2.1 looks impossible.
- **Quoting a graph bound in one parameter.**  $O(|V|^2)$  hides whether you have exploited sparsity;  $O(|V| + |E|)$  says you have.
- **Assuming the average case without saying so.** Especially for sorting, where real inputs are very often partly sorted and the uniform permutation assumption is simply false.
- **Thinking the best case tells you the algorithm is fast.** Insertion sort's best case is  $\Theta(n)$  and it is still a quadratic sort.
- **Confusing “average case” with “expected time”.** The first assumes something about the data; the second assumes nothing. The second is much stronger and is almost always what a randomised algorithm's bound means.

### Try It Yourself: Three Shapes, and the Cost of a Bit

Reproduces both tables in this chapter. Expect twenty-five minutes — part B takes about two minutes of CPU on its own.

```
// ch02_cases.cpp -- the nature of the input, and the size of the input.
// Build: cl /O2 /EHsc /std:c++17 ch02_cases.cpp      (see Appendix A)

// --- 1. Three shapes of input, all of the same size -----
// Best case: already sorted, so the inner while never executes.
// Worst case: reversed, so it runs to the front every time.
// Average: a random permutation, which moves each element about half
// way -- and half of a quadratic is still a quadratic.
static std::vector<int> sorted_input(size_t n) {
    std::vector<int> a(n);
    std::iota(a.begin(), a.end(), 0);
    return a;
}
static std::vector<int> reversed_input(size_t n) {
    std::vector<int> a = sorted_input(n);
    std::reverse(a.begin(), a.end());
    return a;
}

// --- 2. Trial division, whose cost is governed by sqrt(N) -----
// The loop runs about sqrt(N)/2 times when n is prime, which is the
// worst case for this algorithm -- so a prime is what we time.
static bool is_prime_trial(uint64_t n) {
    if (n < 2) return false;
    if (n % 2 == 0) return n == 2;
    for (uint64_t d = 3; d * d <= n; d += 2)
        if (n % d == 0) return false;
    return true;
}

// --- 3. The same algorithm, three orders of growth -----
for (size_t n = 2000; n <= 32000; n *= 2) {
    auto s = sorted_input(n);
    auto r = random_input(n, 12345);
    auto v = reversed_input(n);
    double ts = time_ms([&]{ auto a = s; insertion_sort(a); }, 5);
    double tr = time_ms([&]{ auto a = r; insertion_sort(a); }, 5);
    double tv = time_ms([&]{ auto a = v; insertion_sort(a); }, 5);
    /* print the three times and the three ratios */
}
```

```
// --- 4. Why the size of a number is its number of bits -----
// Each row adds TWO bits -- four more characters of input, if you
// measure input by its value. Watch the time double anyway.
const uint64_t primes[] = {
    4294967291ULL,    17179869143ULL,    68719476731ULL,
    274877906899ULL,  1099511627689ULL,    4398046511093ULL,
    17592186044399ULL, 70368744177643ULL,  281474976710597ULL};
int bits = 32;
for (uint64_t p : primes) {
    bool r = false;
    double t = time_ms([&]{ r = is_prime_trial(p); }, 3);
    if (!r) { printf("  BUG: %llu is not prime\n", p); return 1; }
    printf("%4d %22llu %12.2f %7.2f\n", bits, p, t, t / prev);
    prev = t; bits += 2;
}
```

**What to notice.** In part A, look at the *ratio* of the random column to the reversed column rather than at either alone. It should be close to 0.5, and it is the one number in the experiment that tests the inversion count rather than just the exponent.

**Then try to break it.** Replace the random permutation with an array that is sorted except for the last 100 elements, which are shuffled. Predict the running time before you measure it. This is the input shape real data actually has, and it is the one both the best-case and the average-case analyses describe badly.

### 🔍 Checkpoint — Test Yourself

1. An algorithm takes as input an  $n \times n$  matrix and runs in  $\Theta(n^3)$ . Is it polynomial in the size of its input? Answer in terms of the number of numbers actually read.
2. Linear search for a key that is present at a uniformly random position examines  $(n+1)/2$  elements on average and  $n$  in the worst case. Why does the course nevertheless describe both cases with the same  $\Theta(\cdot)$ ?
3. A colleague reports that their sorting routine is “ $O(n \lg n)$  on average”. What single question would you ask before believing the number means anything for your data?

### 📋 Chapter Summary

- Input size is a measure of how much input there is: elements for an array, the side for a matrix, *two* parameters for a graph, and **bits** for a number.
- A number’s size is its bit length, so a loop running  $N$  times is  $O(2^b)$  — exponential. Measured: trial division doubled for every two bits added, eight times running, mean ratio 2.02.
- Best case promises nothing, worst case promises a guarantee, average case promises something only once you have named the distribution.
- Insertion sort:  $\Theta(n)$  best,  $\Theta(n^2)$  worst,  $\Theta(n^2)$  average — the average being exactly half the worst, which measurement confirmed at 0.54, 0.56, 0.54, 0.50, 0.49.
- At  $n = 32,000$  the same sort took 0.028 ms or 185 ms depending only on the *shape* of its input: a factor of 6,600.
- Report the worst case by default. Report an average only with its distribution attached.
- An average over inputs assumes something about the world; the expected time of a randomised algorithm assumes nothing, which is why Chapter 9 randomises.

 **Review Questions**

1. **(MCQ)** The input size of the integer  $N$  is taken to be: (a)  $N$ ; (b)  $\sqrt{N}$ ; (c) about  $\lg N$ ; (d)  $N^2$ .
2. **(MCQ)** Insertion sort's average-case running time on a uniform random permutation is: (a)  $\Theta(n)$ ; (b)  $\Theta(n \lg n)$ ; (c)  $\Theta(n^2)$ ; (d)  $\Theta(n^2 \lg n)$ .
3. **(Short)** Explain what is meant by the best, expected and worst case behaviour of an algorithm, and give one algorithm whose three cases have three different orders of growth (*CLO-1*).
4. **(Short)** Identify two characteristics of input data that change which case an algorithm exhibits, and name an algorithm affected by each (*CLO-2*).
5. **(Short)** Why is a bound of  $O(|V| + |E|)$  more informative than one of  $O(|V|^2)$ , even when the two are equal for the graph in front of you?
6. **(Applied)** Trial division on a 48-bit prime took 58.81 ms on the book's machine. Estimate the time for a 70-bit prime, state the assumption your estimate rests on, and say why the estimate is more trustworthy than the absolute time it is derived from.
7. **(Applied)** You are given a sort you cannot read the source of. Design an experiment that decides whether its worst case is quadratic, using only the ability to supply inputs and time it. Say what inputs you would supply, what you would compute, and what result would leave you unable to tell.

## Asymptotic Notation

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part I — Analysing an Algorithm

## 🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **State** the formal definitions of  $O$ ,  $\Omega$ ,  $\Theta$ ,  $o$  and  $\omega$ , and **explain** what each one quantifies over (CLO-5).
- **Prove** that a given function belongs to a given asymptotic class by exhibiting the witnesses  $c$  and  $n_0$ , and **disprove** membership by contradiction.
- **Use** the limit test, and **recognise** the case in which it does not apply.
- **Order** the standard functions by growth, and **apply** transitivity and the sum rule to simplify a bound.
- **Identify** the four common abuses of the notation, and **state** what each one conceals.

## 🔗 Before You Start

Chapter 1 and Chapter 2. From Appendix B you need the logarithm identities and, for Section 3.5 only, L'Hôpital's rule.

## 📖 Key Terms in This Chapter

asymptotic upper bound • asymptotic lower bound • tight bound • witness •  $O$  •  $\Omega$  •  $\Theta$  •  $o$  •  $\omega$  • transitivity • transpose symmetry • sum rule • limit test • trichotomy

## 3.1 Three Bounds, Defined

Chapter 1 said we discard constants and keep the dominant term. That is a practice, not a definition, and the practice can be abused. This chapter gives it a definition precise enough to prove things with.

Throughout,  $f$  and  $g$  map the positive integers to the non-negative reals — running times, so never negative.

📑  $O$ ,  $\Omega$  and  $\Theta$ 

**Upper bound.**  $O(g(n))$  is the *set* of functions

$$O(g(n)) = \{ f(n) : \exists c > 0, \exists n_0 > 0, \forall n \geq n_0, 0 \leq f(n) \leq c g(n) \}.$$

**Lower bound.**

$$\Omega(g(n)) = \{ f(n) : \exists c > 0, \exists n_0 > 0, \forall n \geq n_0, 0 \leq c g(n) \leq f(n) \}.$$

**Tight bound.**

$$\Theta(g(n)) = \{ f(n) : \exists c_1, c_2 > 0, \exists n_0 > 0, \forall n \geq n_0, 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \}.$$

The constants  $c$  (or  $c_1, c_2$ ) and  $n_0$  are the **witnesses**. To prove a bound you produce them; to disprove one you show none can exist.

⚠ “ $f(n) = O(g(n))$ ” is an abuse of the equals sign, and you should know it

$O(g)$  is a set, so the honest statement is  $f \in O(g)$ . Everyone writes  $f(n) = O(g(n))$  anyway, including this book, and the convention is harmless as long as you remember that it is **one-directional**:

- $n = O(n^2)$  is true.
- $O(n^2) = n$  is meaningless.
- From  $f = O(n^2)$  and  $h = O(n^2)$  you *cannot* conclude  $f = h$ .

Read the “=” as “is” — as in “Rawalpindi is a city”, which does not make every city Rawalpindi.

### 3.2 Proving a Bound From the Definition

The definitions are existential, so a proof is a construction: name  $c$ , name  $n_0$ , verify the inequality.

🔑 **Theorem 3.1 — A quadratic is  $O(n^2)$**

$$3n^2 + 10n + 7 = O(n^2).$$

*Proof.* We must produce  $c$  and  $n_0$  with  $3n^2 + 10n + 7 \leq cn^2$  for all  $n \geq n_0$ . For  $n \geq 1$  we have  $n \leq n^2$  and  $1 \leq n^2$ , so

$$3n^2 + 10n + 7 \leq 3n^2 + 10n^2 + 7n^2 = 20n^2.$$

Taking  $c = 20$  and  $n_0 = 1$  satisfies the definition. ■

💡 The witnesses are not unique, and need not be tight

That proof produced  $c = 20$ . A sharper argument gives a smaller one: for  $n \geq 10$ ,  $10n + 7 \leq n^2$ , so  $c = 4$  and  $n_0 = 10$  also work.

**Both proofs are equally valid**, because the definition asks only that *some* pair exist. Students waste effort hunting for the smallest  $c$ ; the definition does not reward it. Pick the crudest bound that closes the inequality — here, replacing every term by the largest one.

🧩 **Lemma 3.2 — Polynomials**

Let  $p(n) = \sum_{i=0}^d a_i n^i$  with  $a_d > 0$ . Then  $p(n) = \Theta(n^d)$ .

*Proof. Upper bound.* Let  $A = \sum_{i=0}^d |a_i|$ . For  $n \geq 1$  every  $n^i \leq n^d$ , so  $p(n) \leq An^d$ , giving  $c_2 = A$ .

**Lower bound.** Write  $p(n) = a_d n^d + r(n)$  where  $r(n) = \sum_{i<d} a_i n^i$ . By the same argument  $|r(n)| \leq Bn^{d-1}$  with  $B = \sum_{i<d} |a_i|$ . Hence

$$p(n) \geq a_d n^d - Bn^{d-1} = n^d \left( a_d - \frac{B}{n} \right) \geq \frac{a_d}{2} n^d \quad \text{whenever } n \geq \frac{2B}{a_d}.$$

So  $c_1 = a_d/2$  and  $n_0 = \max\{1, 2B/a_d\}$  complete the pair of bounds, and  $p(n) = \Theta(n^d)$  by definition. ■

» **Corollary 3.3 — Only the degree survives**

The degree of a polynomial running time is the only thing about it that survives asymptotic analysis.  $0.001n^3$  grows faster than  $1000n^2$ .

Disproof is the other direction, and it is a proof by contradiction.

### 🔗 Theorem 3.4 — A quadratic is not linear

$$n^2 \neq O(n).$$

*Proof.* Suppose it were. Then there exist  $c > 0$  and  $n_0 > 0$  such that  $n^2 \leq cn$  for all  $n \geq n_0$ . Since  $n > 0$  we may divide by  $n$ :

$$n \leq c \quad \text{for all } n \geq n_0.$$

But  $c$  is a fixed constant, so taking  $n = \max\{n_0, \lceil c \rceil + 1\}$  gives  $n \geq n_0$  and  $n > c$  simultaneously — contradicting the line above. No such  $c$  exists. ■

### 🔗 Theorem 3.5 — $\Theta$ is exactly $O$ and $\Omega$ together

$$f(n) = \Theta(g(n)) \text{ if and only if } f(n) = O(g(n)) \text{ and } f(n) = \Omega(g(n)).$$

*Proof.* ( $\Rightarrow$ ) Suppose  $f = \Theta(g)$  with witnesses  $c_1, c_2, n_0$ . Then  $f(n) \leq c_2 g(n)$  for  $n \geq n_0$ , which is the definition of  $O$  with  $c = c_2$ ; and  $c_1 g(n) \leq f(n)$  for  $n \geq n_0$ , which is the definition of  $\Omega$  with  $c = c_1$ .

( $\Leftarrow$ ) Suppose  $f = O(g)$  with witnesses  $c_u, n_u$  and  $f = \Omega(g)$  with witnesses  $c_\ell, n_\ell$ . Put  $n_0 = \max\{n_u, n_\ell\}$ . For every  $n \geq n_0$  both inequalities hold at once, so

$$c_\ell g(n) \leq f(n) \leq c_u g(n),$$

which is the definition of  $\Theta$  with  $c_1 = c_\ell$  and  $c_2 = c_u$ . ■

### 💡 Why that theorem earns its place

It is the most-used result in the subject and it is two lines long, so it is tempting to skip. Do not: it is what licenses the standard division of labour in every later analysis.

To establish a  $\Theta$  bound you prove an upper bound and a lower bound *separately*, by completely different arguments. Chapter 10 proves  $\Omega(n \lg n)$  for comparison sorting by a decision-tree argument that has nothing in common with Chapter 8's  $O(n \lg n)$  recurrence — and Theorem 3.5 is the reason the two halves combine into  $\Theta(n \lg n)$ .

## 3.3 The Strict Bounds: $o$ and $\omega$

$O$  allows  $f$  and  $g$  to grow at the same rate. The strict forms forbid it, and the change in the definition is the position of one quantifier.

### 📋 $o$ and $\omega$

$$o(g(n)) = \{ f(n) : \forall c > 0, \exists n_0 > 0, \forall n \geq n_0, 0 \leq f(n) < c g(n) \}$$

$$\omega(g(n)) = \{ f(n) : \forall c > 0, \exists n_0 > 0, \forall n \geq n_0, 0 \leq c g(n) < f(n) \}$$

**The whole difference is  $\exists c$  against  $\forall c$ .** For  $O$  it is enough that *one* constant works. For  $o$ , *every* constant must work, however small — so  $f$  must become negligible relative to  $g$ , not merely bounded by it.

### 🧮 Worked Example: Testing the Quantifier

**Is  $2n^2 = o(n^2)$ ?** Take  $c = 1$ . We would need  $2n^2 < n^2$  for all large  $n$ , i.e.  $2 < 1$ . So **no** — and the reasoning shows why:  $o$  requires the ratio to vanish, and  $2n^2/n^2 = 2$  for every  $n$ .

**Is  $2n = o(n^2)$ ?** Given any  $c > 0$ , we need  $2n < cn^2$ , i.e.  $n > 2/c$ . So  $n_0 = \lceil 2/c \rceil + 1$  works. Since we produced an  $n_0$  for *every*  $c$ , **yes**.

**The pattern.**  $f = o(g)$  exactly when  $f = O(g)$  but  $f \neq \Theta(g)$  — the bound holds and is *not* tight.

### 3.4 Properties, and the Analogy With Inequality

#### Theorem 3.6 — Transitivity

If  $f(n) = O(g(n))$  and  $g(n) = O(h(n))$ , then  $f(n) = O(h(n))$ .

*Proof.* Take witnesses  $c_1, n_1$  for the first and  $c_2, n_2$  for the second, and let  $n_0 = \max\{n_1, n_2\}$ . For  $n \geq n_0$ ,

$$f(n) \leq c_1 g(n) \leq c_1(c_2 h(n)) = (c_1 c_2) h(n),$$

where the second inequality may be multiplied by  $c_1 > 0$  without reversing. So  $c = c_1 c_2$  and  $n_0$  are witnesses for  $f = O(h)$ . The same argument, with the inequalities reversed, gives transitivity for  $\Omega$ , and Theorem 3.5 then gives it for  $\Theta$ . ■

#### Lemma 3.7 — Sum rule

If  $f_1 = O(g_1)$  and  $f_2 = O(g_2)$ , then  $f_1 + f_2 = O(\max\{g_1, g_2\})$ .

*Proof.* With witnesses  $c_1, n_1$  and  $c_2, n_2$ , put  $n_0 = \max\{n_1, n_2\}$  and  $M(n) = \max\{g_1(n), g_2(n)\}$ . For  $n \geq n_0$ ,

$$f_1(n) + f_2(n) \leq c_1 g_1(n) + c_2 g_2(n) \leq c_1 M(n) + c_2 M(n) = (c_1 + c_2) M(n). \quad \blacksquare$$

Lemma 3.7 is why “keep the dominant term” is legitimate: a program that does an  $O(n^2)$  pass and then an  $O(n)$  pass is  $O(n^2)$ , and we do not have to think about it again.

#### The analogy with $\leq$ , and the one place it fails

| Asymptotic      | Reads as   | Properties it really has                                                                           |
|-----------------|------------|----------------------------------------------------------------------------------------------------|
| $f = O(g)$      | $f \leq g$ | transitive; reflexive ( $f = O(f)$ )                                                               |
| $f = \Omega(g)$ | $f \geq g$ | transitive; reflexive                                                                              |
| $f = \Theta(g)$ | $f = g$    | transitive, reflexive <i>and</i> symmetric — an equivalence relation, which the other four are not |
| $f = o(g)$      | $f < g$    | transitive; <i>not</i> reflexive                                                                   |
| $f = \omega(g)$ | $f > g$    | transitive; not reflexive                                                                          |

**Transpose symmetry** also holds:  $f = O(g)$  exactly when  $g = \Omega(f)$ , and  $f = o(g)$  exactly when  $g = \omega(f)$ .

**But trichotomy fails.** For real numbers, exactly one of  $a < b$ ,  $a = b$ ,  $a > b$  holds. For functions there is a fourth possibility: *incomparable*. Let

$$f(n) = n^{1+\sin n}.$$

As  $n$  runs over the integers  $\sin n$  oscillates densely in  $[-1, 1]$ , so the exponent oscillates in  $[0, 2]$ . Infinitely often  $f(n)$  is near  $n^0 = 1$ , which kills any  $\Omega(n)$  bound; infinitely often it is near  $n^2$ , which kills any  $O(n)$  bound. So  $f$  is **neither**  $O(n)$  **nor**  $\Omega(n)$  — the two functions simply cannot be ordered.

Pathological, and worth one paragraph: it is the reason the next section's limit test needs a disclaimer, and the reason you may not argue “ $f$  is not  $O(g)$ , therefore  $f$  is  $\Omega(g)$ ”.

### 3.5 The Limit Test

Constructing witnesses is reliable and slow. For most functions met in practice, a limit settles the question at once.

#### Theorem 3.8 — Limit test

Suppose  $L = \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$  exists, with  $L = \infty$  allowed. Then

$$L = 0 \implies f = o(g); \quad 0 < L < \infty \implies f = \Theta(g); \quad L = \infty \implies f = \omega(g).$$

*Proof.* We prove the middle case; the others are similar. Let  $0 < L < \infty$ . By the definition of a limit, taking  $\varepsilon = L/2$  there is an  $n_0$  such that for all  $n \geq n_0$ ,

$$\left| \frac{f(n)}{g(n)} - L \right| < \frac{L}{2} \implies \frac{L}{2} < \frac{f(n)}{g(n)} < \frac{3L}{2}.$$

Multiplying through by  $g(n) > 0$  gives  $\frac{L}{2}g(n) < f(n) < \frac{3L}{2}g(n)$ , so  $c_1 = L/2$  and  $c_2 = 3L/2$  witness  $f = \Theta(g)$ . ■

#### The test is one-directional

Theorem 3.8 assumes the limit *exists*. If it does not, the test says nothing at all — it does not say the functions are incomparable.

For  $f(n) = n^{1+\sin n}$  and  $g(n) = n$  the ratio  $n^{\sin n}$  oscillates forever and the limit does not exist; here the functions do turn out to be incomparable. But  $f(n) = n(2 + \sin n)$  also has no limiting ratio against  $g(n) = n$ , and yet  $f = \Theta(n)$  plainly, with  $c_1 = 1$  and  $c_2 = 3$ . When the limit fails, go back to the witnesses.

#### Worked Example: Logarithms Lose to Every Power

**Claim:**  $\lg n = o(n^\varepsilon)$  for every constant  $\varepsilon > 0$  — however small.

**Step 1.** Both  $\lg n \rightarrow \infty$  and  $n^\varepsilon \rightarrow \infty$ , so the ratio is of the form  $\infty/\infty$  and L'Hôpital's rule applies. Treat  $n$  as a real variable  $x$  and use  $\lg x = \ln x / \ln 2$ :

$$\lim_{x \rightarrow \infty} \frac{\lg x}{x^\varepsilon} = \lim_{x \rightarrow \infty} \frac{(1/(x \ln 2))}{\varepsilon x^{\varepsilon-1}} = \lim_{x \rightarrow \infty} \frac{1}{\varepsilon \ln 2 \cdot x^\varepsilon} = 0.$$

**Step 2.** The limit is 0, so by Theorem 3.8,  $\lg n = o(n^\varepsilon)$ .

**Step 3 — what it means.** Take  $\varepsilon = 0.01$ . The claim is that  $\lg n$  is eventually negligible beside  $n^{0.01}$ , a function so flat it looks constant on any plot you can draw. “Eventually” is doing heavy work: at  $n = 10^6$ ,  $\lg n \approx 20$  while  $n^{0.01} \approx 1.15$ , so the inequality has not begun to hold. It holds from about  $n = 10^{530}$ .

**Step 4 — the lesson.** Asymptotic statements are about limits, and a limit carries no promise about  $n = 10^6$ . This is the same lesson as Chapter 1’s crossover at  $n = 65$ , in a more extreme form.

#### 💡 The hierarchy worth memorising

Each function below is  $o$  of the one after it, for any constants  $\varepsilon > 0$ ,  $k \geq 1$  and  $a > 1$ :

$$1 \prec \lg \lg n \prec \lg n \prec (\lg n)^k \prec n^\varepsilon \prec n \prec n \lg n \prec n^2 \prec n^k \prec a^n \prec n!$$

At  $n = 10^6$ , taking one operation per nanosecond:

| $g(n)$    | Operations        | Time                          |
|-----------|-------------------|-------------------------------|
| $\lg n$   | 20                | 20 nanoseconds                |
| $n$       | $10^6$            | 1 millisecond                 |
| $n \lg n$ | $2.0 \times 10^7$ | 20 milliseconds               |
| $n^2$     | $10^{12}$         | 17 minutes                    |
| $n^3$     | $10^{18}$         | 32 years                      |
| $2^n$     | —                 | no physical comparison exists |

The gap from  $n \lg n$  to  $n^2$  — 20 milliseconds to 17 minutes — is the one most algorithm design is about.

### 3.6 What $\Theta$ Asserts, Measured

$f = \Theta(g)$  says  $f(n)/g(n)$  is eventually trapped between two positive constants. That is a claim about a ratio, so it can be tested directly — and tested against *rival* exponents, which is the sharper experiment.

#### 📊 Worked Example: Ruling Out the Exponents Either Side

`code/ch03_fit.cpp` times insertion sort and divides by three candidate functions. Each column is normalised to 1.000 at the first row, because the constant itself is machine-specific and the question is only whether the column *stays put*. Median of nine runs.

| $n$     | $T$ (ms) | $T/n^{1.9}$ | $T/n^{2.0}$ | $T/n^{2.1}$ |
|---------|----------|-------------|-------------|-------------|
| 4,000   | 2.45     | 1.000       | 1.000       | 1.000       |
| 8,000   | 10.22    | 1.116       | 1.041       | 0.972       |
| 16,000  | 39.19    | 1.146       | 0.998       | 0.869       |
| 32,000  | 162.82   | 1.276       | 1.036       | 0.842       |
| 64,000  | 678.31   | 1.424       | 1.079       | 0.818       |
| 128,000 | 2595.28  | 1.460       | 1.032       | 0.730       |

Read the three columns as three hypotheses.

- $T/n^{1.9}$  rises by 46%, without turning over. If  $T = \Theta(n^{1.9})$  this column would be trapped between constants; it is climbing, so  $n^{1.9}$  is too small.
- $T/n^{2.0}$  stays inside  $[0.998, 1.079]$  — a spread of 8% across a 32-fold range of  $n$ . That is a ratio trapped between two constants, which is exactly the definition.
- $T/n^{2.1}$  falls by 27%, monotonically. So  $n^{2.1}$  is too large:  $T = o(n^{2.1})$ , an upper bound that is not tight.

**The experiment therefore does more than confirm the exponent** — it excludes the two exponents either side, which is what distinguishes evidence for  $\Theta(n^2)$  from mere consistency with it.

#### 📊 Worked Example: The Same Test for $n \lg n$ , Which Is Harder

$n \lg n$  sits only a whisker above  $n$ , so this test needs both a wider range of  $n$  and more careful timing.

| $n$       | $T$ (ms) | $T/n$ | $T/(n \lg n)$ | $T/n^{1.2}$ |
|-----------|----------|-------|---------------|-------------|
| 100,000   | 11.49    | 1.000 | 1.000         | 1.000       |
| 200,000   | 23.89    | 1.040 | 0.980         | 0.905       |
| 400,000   | 52.63    | 1.145 | 1.022         | 0.868       |
| 800,000   | 107.19   | 1.166 | 0.988         | 0.769       |
| 1,600,000 | 237.89   | 1.294 | 1.043         | 0.743       |
| 3,200,000 | 489.47   | 1.331 | 1.023         | 0.666       |
| 6,400,000 | 1036.62  | 1.410 | 1.036         | 0.614       |

$T/(n \lg n)$  stays within 4.3% of 1 over a 64-fold range of  $n$ . The rivals both fail:  $T/n$  climbs 41%,  $T/n^{1.2}$  falls 39%.

**Now the detail that makes this more than a curve fit.** If  $T$  really is  $cn \lg n$ , then  $T/n = c \lg n$ , so the  $T/n$  column should itself grow like  $\lg n$ . Over this range that predicts a rise of

$$\frac{\lg(6,400,000)}{\lg(100,000)} = \frac{22.61}{16.61} = 1.361.$$

Measured: **1.410**, within 4% of the prediction. The drift in the failing column is not noise — it is the  $\lg n$  factor, measured.

**And what it took to get there.** The first draft of this program allocated a fresh vector inside the timed region. At these sizes the allocation cost more than the sort it was supposed to be measuring, and the  $T/(n \lg n)$  column came out as 1.000, 1.134, 0.804, 1.342, 0.839 — noise, from which nothing follows. The fix was to allocate once outside the loop, refill with `std::copy`, and *measure the refill separately and subtract it*; the `copy` column in the program's output is that measurement, and it is how you can tell the subtraction was small enough to trust. Appendix A collects the rest of these rules.

### 3.7 Four Abuses, and What Each Conceals

**⚠ 1. Using  $O$  when you mean  $\Theta$** 

“Insertion sort is  $O(n^2)$ ” is true. So is “insertion sort is  $O(n^{100})$ ”, and so is “ $O(2^n)$ ” — an upper bound stays true however loose it gets. If you know the bound is tight, say  $\Theta$  and claim the credit; if you do not,  $O$  is honest and you should expect to be asked.

The reverse error is worse: reading somebody’s  $O(\cdot)$  as if it were a  $\Theta(\cdot)$ , and concluding an algorithm is slower than it is.

**⚠ 2.  $\Omega$  as an excuse**

“My algorithm is  $\Omega(n)$ ” is a boast about a *lower* bound, which for an algorithm is bad news, not good. Lower bounds are interesting about *problems* — “every comparison sort is  $\Omega(n \lg n)$ ”, Chapter 10 — because they say no algorithm can do better. About your own code,  $\Omega(\cdot)$  means “at least this slow”.

**⚠ 3. Dropping the case**

$\Theta(\cdot)$  says nothing about which input you had in mind. Quicksort is  $\Theta(n^2)$  in the worst case and  $\Theta(n \lg n)$  in expectation; both are true and they differ by a factor of thousands. **Name the case every time** — the notation has no slot for it, so the sentence must.

**⚠ 4. Forgetting that the constants went somewhere**

Asymptotic notation deliberately discards exactly the information that decides which program to run at your actual  $n$ . From `ch03_fit.cpp`:  $100n \lg n < n^2$  first holds at  $n = 997$  — not at  $n = 10$ . Below a thousand elements the quadratic algorithm wins, and  $\Theta(\cdot)$  cannot tell you that.

The notation is a tool for comparing *ideas*. Choosing an *implementation* needs the constants back, which means measuring — the other half of every chapter in this book.

**! Common Pitfalls**

- **Writing  $O(2n^2)$  or  $O(n^2 + n)$ .** Legal but illiterate: the point of the notation is that constants and lower terms are already gone. Write  $O(n^2)$ .
- **Hunting for the smallest  $c$ .** The definition asks for existence. Any witness pair is a complete proof.
- **Concluding  $f = \Omega(g)$  from “ $f \neq O(g)$ ”.** Trichotomy fails;  $n^{1+\sin n}$  is the counterexample.
- **Using the limit test without checking the limit exists.** When it does not, the test is silent, not negative.
- **Comparing  $O(\cdot)$  bounds to rank two algorithms.** Both being  $O(n^2)$  tells you nothing about which is faster, nor even that either is quadratic.
- **Saying “ $\lg n$  is basically constant”.** It is  $o(n^\varepsilon)$  for every  $\varepsilon$ , and it is also 20 at  $n = 10^6$  and 30 at  $n = 10^9$ . Negligible in the limit is not the same as negligible.
- **Timing inside an allocation.** Not a notation error, but the reason the second worked example nearly reported nonsense.

**🔧 Try It Yourself: Fitting a Curve, and Excluding the Rivals**

Reproduces both tables in Section 3.6 and the crossover figure. Expect thirty minutes; part A takes about a minute of CPU at  $n = 128,000$ .

```
// ch03_fit.cpp -- what Theta(g) actually asserts, tested.
// Build: cl /O2 /EHsc /std:c++17 ch03_fit.cpp           (see Appendix A)
```

```

// --- 1. Insertion sort against three candidate exponents -----
// Only one of the three columns can stay constant, and which one
// does is the empirical content of "Theta(n^2)".
//
// Normalising to the FIRST row makes that row's noise everybody's
// noise, so the first row must not be a sub-millisecond timing:
// start at n = 4000 and take the median of nine runs.
double b19 = 0, b20 = 0, b21 = 0;
for (size_t n = 4000; n <= 128000; n *= 2) {
    auto base = random_input(n, 2024);
    std::vector<int> work(n);
    double t = time_ms([&]{
        std::copy(base.begin(), base.end(), work.begin());
        insertion_sort(work);
    }, 9);
    double x = (double)n;
    double r19 = t / std::pow(x, 1.9);
    double r20 = t / std::pow(x, 2.0);
    double r21 = t / std::pow(x, 2.1);
    if (b19 == 0) { b19 = r19; b20 = r20; b21 = r21; }
    printf("%8zu %10.2f %10.3f %10.3f %10.3f\n",
        n, t, r19/b19, r20/b20, r21/b21);
}

// --- 2. Merge sort against n*lg(n) and its two rivals -----
// n*lg(n) is only a whisker above n, so this is a far harder test
// than part 1, and it needs better hygiene to pass.
//
// The first draft allocated a fresh vector inside the timed region.
// At these sizes that allocation costs more than it measures -- the
// T/(n lg n) column came out as 1.000, 1.134, 0.804, 1.342, 0.839,
// which says nothing at all.
//
// The fix: allocate ONCE outside the timing and refill with
// std::copy, then measure the refill alone and subtract it.
for (size_t n = 100000; n <= 6400000; n *= 2) {
    auto base = random_input(n, 2024);
    std::vector<int> work(n), buf(n);
    double tc = time_ms([&]{
        std::copy(base.begin(), base.end(), work.begin());
    }, 11);
    double tt = time_ms([&]{
        std::copy(base.begin(), base.end(), work.begin());
        merge_sort(work, buf, 0, n);
    }, 11);
    double t = tt - tc; // the sort, with the refill removed
    /* print tc so the reader can check the subtraction was small,
       then t/n, t/(n lg n) and t/n^1.2 normalised to the first row */
}

// --- 3. Where an asymptotically worse function is still smaller ----
// Theta is a statement about a limit, and the limit can be a long way
// off. Report the crossover rather than pretending there is none:
// 100 n lg n < n^2 requires 100 lg n < n.
for (size_t n = 2; n < 100000; ++n) {
    double x = (double)n;
    if (100.0 * x * std::log2(x) < x * x) {
        printf("\n    crossover: n lg n first wins at n = %zu\n", n);
        break;
    }
}

```

**What to notice.** The `copy` column in part 2 is there to be audited, not admired: if it ever approaches the `total` column, the subtraction is doing too much work and the experiment should be redesigned rather than reported.

**Then try to break it.** Add a  $T/n^{2.05}$  column to part 1. It will drift down, but only slightly — and that is the honest limit of this method. Timing can separate  $n^{1.9}$  from  $n^{2.1}$  over a 32-fold range; it cannot separate  $n^2$  from  $n^{2.02}$ , and no amount of care will make it. Say what an experiment can resolve before you claim a result from it.

### ? Checkpoint — Test Yourself

1. Prove that  $5n^3 + 200n^2 + 17 = O(n^3)$  by exhibiting witnesses  $c$  and  $n_0$ , and verify your inequality at  $n = n_0$ .
2. Is  $n^2 = o(n^2 \lg n)$ ? Justify your answer from the definition, not from the limit test.
3. A student writes: “my algorithm is  $O(n^3)$  and my colleague’s is  $O(n^2)$ , so mine is slower.” Give the two distinct reasons this conclusion does not follow.

### ✓ Chapter Summary

- $O$ ,  $\Omega$  and  $\Theta$  are *sets* of functions, defined by the existence of witnesses  $c$  and  $n_0$ . “ $f = O(g)$ ” means  $f \in O(g)$  and may not be read backwards.
- To prove a bound, exhibit witnesses — any witnesses. To disprove one, derive a contradiction, as for  $n^2 \neq O(n)$ .
- $f = \Theta(g)$  iff  $f = O(g)$  and  $f = \Omega(g)$ , which is what lets upper and lower bounds be proved by unrelated arguments and combined.
- $o$  and  $\omega$  differ from  $O$  and  $\Omega$  only in having  $\forall c$  where those have  $\exists c$ , which forces the ratio to vanish or diverge.
- All five are transitive; only  $\Theta$  is an equivalence relation; and trichotomy fails —  $n^{1+\sin n}$  is comparable to  $n$  in no direction at all.
- The limit test decides the question whenever the limit exists, and is silent, not negative, when it does not.
- Measured:  $T/n^{2.0}$  for insertion sort held within 8% across a 32-fold range of  $n$  while  $T/n^{1.9}$  rose 46% and  $T/n^{2.1}$  fell 27%. For merge sort  $T/(n \lg n)$  held within 4.3% across a 64-fold range, and the drift in the failing  $T/n$  column matched the predicted  $\lg n$  growth to within 4%.
- The notation discards precisely what decides which program to run:  $100n \lg n$  does not beat  $n^2$  until  $n = 997$ .

### 🔧 Review Questions

1. (MCQ)  $f(n) = \Theta(g(n))$  holds if and only if: (a)  $f = O(g)$ ; (b)  $f = \Omega(g)$ ; (c) both of these; (d)  $f = o(g)$ .
2. (MCQ) Which is *false*? (a)  $n = O(n^2)$ ; (b)  $n^2 = O(n)$ ; (c)  $2^{n+1} = O(2^n)$ ; (d)  $\lg n = o(n)$ .
3. (Short) State the formal definition of  $O(g(n))$  and explain the role of each quantifier (CLO-5).
4. (Short) Explain the difference between  $O(g)$  and  $o(g)$  in terms of the quantifier on  $c$ , and give a function that is in the first but not the second.
5. (Short) Why is a lower bound a desirable result about a problem but an undesirable one about your own algorithm?
6. (Applied) Prove  $\Theta(\cdot)$  is transitive, using Theorems 3.5 and 3.6 rather than arguing from the definition directly.

7. (**Applied**) You time an unknown algorithm and find that  $T(n)/n^2$  falls steadily while  $T(n)/(n \lg n)$  rises steadily across your whole range of  $n$ . What can you conclude, what can you not, and what would you measure next?

## Correctness: Loop Invariants

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part I — Analysing an Algorithm

### 🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** why testing cannot establish that an algorithm is correct, and **quantify** the input space it would have to cover.
- **State** a loop invariant for a given loop, and **name** the three obligations a proof must discharge.
- **Prove** insertion sort and binary search correct from their invariants, including the permutation clause.
- **Distinguish** partial correctness from total correctness, and **prove** termination by exhibiting a decreasing measure.
- **Diagnose** a loop whose invariant holds but which fails to terminate.

### 🔗 Before You Start

Chapter 1 for the vocabulary. Section 4.2 leans on mathematical induction; Appendix B has it if you want a refresher first.

### 📖 Key Terms in This Chapter

correctness • partial correctness • total correctness • loop invariant • initialisation • maintenance • termination • precondition • postcondition • permutation • loop variant • decreasing measure • well-founded

## 4.1 Why Testing Is Not Enough

A sorting routine takes any array of any length holding any values. That is an infinite set of inputs, and a test suite is a finite list. The gap between them is not a matter of diligence; it is a matter of cardinality.

### 🧮 Worked Example: Counting What Exhaustive Testing Would Cost

Restrict the problem absurdly: arrays of exactly  $n$  elements, each value drawn from 0 to 99. How many distinct sorted inputs must a binary search be tested against?

**Step 1.** Only the multiset of values matters, so the count is the number of multisets of size  $n$  from an alphabet of  $m = 100$ :

$$\binom{m+n-1}{n}.$$

**Step 2.** Evaluated by `code/ch04_invariant.cpp`, and each array must then be tried against all 100 possible keys:

| $n$ | sorted arrays         | $\times$ keys        |
|-----|-----------------------|----------------------|
| 4   | $4.42 \times 10^6$    | $4.4 \times 10^8$    |
| 8   | $3.26 \times 10^{11}$ | $3.3 \times 10^{13}$ |
| 16  | $1.50 \times 10^{19}$ | $1.5 \times 10^{21}$ |
| 20  | $2.46 \times 10^{22}$ | $2.5 \times 10^{24}$ |

**Step 3.** At a billion tests a second, the  $n = 20$  row takes  $2.5 \times 10^{15}$  seconds — about **80 million years**. And that is for arrays of twenty elements holding two-digit numbers.

**Step 4 — the conclusion.** Testing samples the input space; it cannot cover it. A proof covers it, in half a page, for every  $n$  and every alphabet at once. The two activities answer different questions, and this chapter is about the one testing cannot answer.

#### 💡 What testing is still for

None of this says do not test. A proof establishes that the *algorithm* is correct; it says nothing about whether your *program* implements that algorithm, which is where most real defects live — an off-by-one in a loop bound, a wrong variable name, an integer overflow the model ignored.

Proof and testing catch disjoint classes of bug. The lab in this chapter contains a defect that *testing found in a third of its trials and a careless proof would have missed*, and Chapter 16's lab contains the reverse. Use both.

## 4.2 The Loop Invariant

### 📖 Loop Invariant

A **loop invariant** is an assertion about the program's state that is true immediately before the loop's first iteration and immediately before every subsequent one.

A proof by loop invariant has **three obligations**:

1. **Initialisation.** The invariant is true before the first iteration.
2. **Maintenance.** If it is true before an iteration, it is true before the next one.
3. **Termination.** The loop ends, and when it does the invariant — together with the reason the loop stopped — gives the result you wanted.

#### 💡 It is induction, wearing different clothes

| Invariant proof | Induction      | On what                                                                                            |
|-----------------|----------------|----------------------------------------------------------------------------------------------------|
| initialisation  | base case      | the loop has run 0 times                                                                           |
| maintenance     | inductive step | assume after $k$ , show after $k + 1$                                                              |
| termination     | —              | no counterpart: induction proves a property for all $k$ , and says nothing about the loop stopping |

**The missing row is the whole point of Section 4.5.** Induction on the iteration count gives you “the invariant holds after every iteration that happens”. Whether any particular iteration happens is a different question, and nothing in the first two obligations answers it.

### Partial and Total Correctness

An algorithm is **partially correct** if, *whenever it halts*, its output satisfies the postcondition. It is **totally correct** if it is partially correct *and* halts on every input satisfying the precondition.

Initialisation and maintenance establish partial correctness. Termination is what upgrades it, and Section 4.6 measures a program that has the first and not the second.

## 4.3 Insertion Sort, Proved

### INSERTION-SORT( $A, n$ )

```

1 for  $j \leftarrow 2$  to  $n$  do
2    $key \leftarrow A[j]$   $\triangleright$  the element to place
3    $i \leftarrow j - 1$ 
4   while  $i > 0$  and  $A[i] > key$  do
5      $A[i + 1] \leftarrow A[i]$   $\triangleright$  shift right
6      $i \leftarrow i - 1$ 
7   end while
8    $A[i + 1] \leftarrow key$ 
9 end for

```

**Figure 4.1:** INSERTION-SORT. The outer loop's invariant concerns the prefix  $A[1..j-1]$ ; the inner loop's concerns where  $key$  has got to. Lines are numbered so the proof can cite them.

The algorithm is indexed from 1, as the pseudocode throughout this book is, and the C++ in the labs is indexed from 0. Appendix A explains the translation; the mathematics is cleaner from 1 and the code is cleaner from 0, and pretending otherwise helps nobody.

### Theorem 4.1 — Insertion sort is correct

Given an array  $A[1..n]$  of  $n \geq 1$  numbers, INSERTION-SORT terminates with  $A$  holding a permutation of its original contents, in non-decreasing order.

*Proof.* Take as the invariant for the **for** loop:

*At the start of each iteration with index  $j$ , the subarray  $A[1..j-1]$  consists of the elements originally in  $A[1..j-1]$ , in non-decreasing order.*

**Initialisation.** Before the first iteration  $j = 2$ , so the subarray is  $A[1..1]$ : one element, trivially sorted, and trivially the element originally there.

**Maintenance.** Assume the invariant before the iteration with index  $j$ . The inner **while** loop shifts  $A[i]$  to  $A[i+1]$  for decreasing  $i$  while  $A[i] > key$ , maintaining its own invariant: *the entries  $A[i+2..j]$  are those originally in  $A[i+1..j-1]$ , all of them  $> key$ , and position  $A[i+1]$  is free.* The loop exits when  $i = 0$  or  $A[i] \leq key$ ; in either case every entry left of  $i+1$  is  $\leq key$  and every entry right of it is  $> key$ , so writing  $key$  into the free slot  $A[i+1]$  leaves  $A[1..j]$  sorted. No element was overwritten without first being copied, and  $key$  was saved before the shifting began, so the subarray still holds exactly the original elements. Incrementing  $j$  makes this the invariant for the next iteration.

**Termination.** The loop variable  $j$  increases by one each iteration and the loop ends when  $j = n + 1$ . Substituting into the invariant:  $A[1..n]$  consists of the elements originally in  $A[1..n]$ , in non-decreasing order — which is the postcondition. Termination itself is immediate because  $j$

strictly increases towards the fixed bound  $n + 1$ , and the inner loop terminates because  $i$  strictly decreases towards the guard  $i > 0$ . ■

#### ⚠ Do not drop the permutation clause

It is tempting to state the invariant as “ $A[1..j-1]$  is sorted” and save eleven words. Then consider this implementation:

*overwrite every element of  $A$  with zero.*

Its output is sorted. It satisfies the shortened invariant at every iteration and terminates. It is not a sorting algorithm.

“Sorted” is only half the postcondition; “a permutation of the input” is the other half, and a proof that omits it proves the wrong theorem. This is the single most common error in student invariant proofs.

### 4.4 Binary Search, Where the Invariant Is About Exclusion

Insertion sort’s invariant describes what has been *built*. The other common shape describes what has been *ruled out*.

≡ BINARY-SEARCH( $A, n, x$ )

```

1  $lo \leftarrow 1$ ;  $hi \leftarrow n$ 
2 while  $lo \leq hi$  do
3    $mid \leftarrow \lfloor (lo + hi)/2 \rfloor$ 
4   if  $A[mid] = x$  then
5     return  $mid$ 
6   else if  $A[mid] < x$  then
7      $lo \leftarrow mid + 1$  ▷ left half cannot hold  $x$ 
8   else
9      $hi \leftarrow mid - 1$  ▷ right half cannot hold  $x$ 
10  end if
11 end while
12 return NOT-FOUND

```

**Figure 4.2:** BINARY-SEARCH on a sorted array. The  $+1$  on line 7 and the  $-1$  on line 9 are what make the search range strictly smaller every iteration  
— Section 4.5 is about what happens when one of them is missing.

#### 🔑 Theorem 4.2 — Binary search is correct

Let  $A[1..n]$  be sorted in non-decreasing order. BINARY-SEARCH returns an index  $mid$  with  $A[mid] = x$  if  $x$  occurs in  $A$ , and NOT-FOUND otherwise.

*Proof.* Invariant: *if  $x$  occurs anywhere in  $A$ , then it occurs in  $A[lo..hi]$ .*

**Initialisation.**  $lo = 1$  and  $hi = n$ , so the range is all of  $A$  and the implication is trivial.

**Maintenance.** Suppose the invariant holds and the loop body runs. If  $A[mid] = x$  the procedure returns a correct index and there is nothing to maintain. If  $A[mid] < x$  then, because  $A$  is sorted, every index  $k \leq mid$  has  $A[k] \leq A[mid] < x$ , so no such position holds  $x$ ; discarding them by setting  $lo \leftarrow mid + 1$  therefore cannot discard an occurrence of  $x$ , and the invariant survives. The case  $A[mid] > x$  is symmetric.

**Termination.** Each iteration either returns, or replaces  $hi - lo$  by a strictly smaller value: line 7 raises  $lo$  past  $mid \geq lo$ , and line 9 lowers  $hi$  below  $mid \leq hi$ . Since  $hi - lo$  is an integer that strictly decreases and the loop runs only while  $hi - lo \geq 0$ , the loop performs finitely many iterations. If it exits, then  $lo > hi$ , so the range  $A[lo..hi]$  is empty; by the invariant  $x$  occurs in  $A$  only if it occurs in an empty range, so it does not occur at all, and NOT-FOUND is correct. ■

#### » Corollary 4.3 — Binary search is logarithmic

Binary search performs at most  $\lfloor \lg n \rfloor + 1$  iterations, since  $hi - lo + 1$  at least halves each time and the loop stops when it reaches 0.

### 4.5 Termination: The Obligation People Skip

The termination argument above did real work, and it is worth isolating what kind of work.

#### ≡ Loop Variant

A **loop variant**, or decreasing measure, is a quantity  $\mu$  computed from the program state such that

- $\mu$  takes values in a **well-founded** set — for our purposes, the non-negative integers;
- every iteration strictly decreases  $\mu$ .

Then the loop terminates, because a strictly decreasing sequence of non-negative integers must be finite.

The variants used so far:  $n + 1 - j$  for insertion sort's outer loop,  $i$  for its inner loop, and  $hi - lo + 1$  for binary search.

#### ≡ EUCLID-GCD( $a, b$ )

**Require:**  $a \geq 0, b \geq 0$ , not both zero

```

1 while  $b \neq 0$  do
2    $r \leftarrow a \bmod b$ 
3    $a \leftarrow b$ 
4    $b \leftarrow r$ 
5 end while
6 return  $a$ 
```

**Figure 4.3:** EUCLID-GCD. Here the invariant is an equality —  $\gcd(a, b)$  never changes — and the variant is simply  $b$ , whose decrease is what makes the loop finite.

#### ♀ Theorem 4.4 — Euclid's algorithm is totally correct

EUCLID-GCD( $a, b$ ) terminates and returns  $\gcd(a, b)$ .

*Proof.* **Invariant:**  $\gcd(a, b)$  equals the greatest common divisor of the original arguments, and not both of  $a, b$  are zero.

**Initialisation.** Immediate —  $a$  and  $b$  are the original arguments.

**Maintenance.** The body replaces  $(a, b)$  by  $(b, a \bmod b)$ . Write  $a = qb + r$  with  $r = a \bmod b$ . Any  $d$  dividing both  $b$  and  $r$  divides  $qb + r = a$ ; any  $d$  dividing both  $a$  and  $b$  divides  $a - qb = r$ . So the pairs  $(a, b)$  and  $(b, r)$  have the same set of common divisors, hence the same greatest one, and the invariant is preserved.

**Termination.** Take  $\mu = b$ . Since  $b \neq 0$  in the body,  $r = a \bmod b$  satisfies  $0 \leq r < b$ , so the new value of  $b$  is strictly less than the old one and never negative. A strictly decreasing sequence of non-negative integers is finite, so the loop ends — with  $b = 0$ . Then  $\gcd(a, 0) = a$ , and by the invariant  $a$  is the gcd of the original arguments, which is what line 6 returns. ■

⚠ Where the measure must live

“Strictly decreasing” is not enough on its own; the measure must decrease within a *well-founded* set. Over the non-negative reals it is not:

$$x \leftarrow x/2 \quad \text{from } x = 1$$

strictly decreases forever and never terminates. Over the non-negative integers no such sequence exists, which is the entire force of the argument.

Use integers, and if your measure is naturally a real number, find an integer that bounds it.

4.6 A Loop Whose Invariant Holds and Which Never Finishes

Here is the case the three obligations exist to catch. Change one character of BINARY-SEARCH: on line 7 write  $lo \leftarrow mid$  instead of  $lo \leftarrow mid + 1$ .

💡 The invariant still holds — and that is the problem

Re-read the maintenance argument of Theorem 4.2. It establishes that positions  $k \leq mid$  cannot hold  $x$ , and therefore that discarding them is safe. Setting  $lo \leftarrow mid$  discards *fewer* positions than that — it keeps  $mid$  itself. Keeping a position cannot lose an occurrence of  $x$ , so:

**Initialisation still holds. Maintenance still holds.** The modified program is *partially correct*: if it answers, the answer is right.

**Termination fails.** When  $hi = lo + 1$  we get  $mid = \lfloor (2lo + 1)/2 \rfloor = lo$ , so  $lo \leftarrow mid$  leaves  $lo$  and  $hi$  unchanged and the next iteration is identical. The measure  $hi - lo$  no longer strictly decreases, and the loop spins forever.

📊 Worked Example: Measured: 132 Million Iterations, Zero Wrong Answers

code/ch04\_invariant.cpp runs the broken search on a 64-element sorted array, 10,000 trials with keys known to be present and 10,000 with keys known to be absent, and *audits the invariant on every single iteration*. A run that exceeds 10,000 iterations is recorded as non-terminating.

| 10,000 trials each   | non-termination | wrong answer |
|----------------------|-----------------|--------------|
| keys always present  | 3,228           | 0            |
| keys always absent   | 10,000          | 0            |
| iterations audited   | 132,311,650     |              |
| invariant violations | 0               |              |

**Read the second column first.** In 20,000 trials the broken search returned a wrong answer **not once**. That is not luck; it is precisely what the invariant guarantees, and the audit confirms the invariant never broke across 132 million iterations.

**Now read the first column.** 13,228 of the 20,000 runs never finished. The program is partially correct and totally useless.

**The distribution matters for how fast you find it.** Keys drawn from inside the array hang about a third of the time; keys known to be absent hang every time. A test suite built only from present keys would still have caught this — in roughly three trials — which is worth saying plainly, because the tidier story would be that testing misses it. Testing found this bug easily.

**What testing could not have told you is *why*,** or that the defect is confined entirely to termination with partial correctness intact. And a proof that checked only initialisation and maintenance — which is what most students write — would have certified this program as correct.

### ! Common Pitfalls

- **Omitting the permutation clause** from a sorting invariant. The all-zeros “sort” satisfies the weakened version.
- **Stating an invariant that mentions no loop variable.** “A is an array of numbers” is true throughout and proves nothing. A useful invariant relates the work done so far to the loop index.
- **Checking the invariant at the wrong moment.** It is asserted *before* each iteration, not in the middle of the body, where it is routinely false — insertion sort’s prefix is genuinely unsorted while the shifting is in progress.
- **Treating termination as a formality.** Section 4.6 is a program that passes the other two obligations and runs forever.
- **A measure that decreases but not in a well-founded set.**  $x \leftarrow x/2$  over the reals.
- **Proving the loop correct and forgetting the exit condition.** The invariant alone is not the postcondition; you need the invariant *and* the reason the loop stopped, which is why every termination paragraph above substitutes the final index back in.
- **Assuming a proved algorithm gives a correct program.** The proof is about the algorithm; overflow, off-by-one and typing errors live below it.

### 🔧 Try It Yourself: A Bug That Is Never Wrong

Reproduces the table in Section 4.6. Expect twenty-five minutes; part C audits 132 million iterations and takes about a minute.

```
// ch04_invariant.cpp -- why the THIRD obligation is not decoration.
// Build: cl /O2 /EHsc /std:c++17 ch04_invariant.cpp (see Appendix A)

// --- 1. The correct version, and the invariant it maintains -----
// Invariant: if x occurs in a, then it occurs in a[lo..hi].
// Initialisation: lo = 0, hi = n-1, so the range is all of a.
// Maintenance:   a is sorted, so discarding the half that cannot
//                contain x preserves the invariant.
// Termination:   hi - lo strictly decreases and is bounded below.
static int bsearch_ok(const std::vector<int>& a, int x) {
    int lo = 0, hi = (int)a.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == x) return mid;
        if (a[mid] < x) lo = mid + 1;    // the correct step
        else           hi = mid - 1;
    }
    return -1;
}

// --- 2. The same search with one character changed -----
// "lo = mid" still discards nothing that could hold x, so the
```

```

// invariant SURVIVES. But when hi == lo + 1 and a[mid] < x, mid
// computes to lo, so lo = mid leaves lo and hi UNCHANGED and the loop
// makes no progress. It is not wrong; it is non-terminating.
static int bsearch_bug(const std::vector<int>& a, int x,
                      bool* inv_ever_broken, long long* iters) {
    int lo = 0, hi = (int)a.size() - 1;
    for (int step = 0; step < CAP; ++step) {
        if (lo > hi) return -1;
        int mid = lo + (hi - lo) / 2;

        // Audit the invariant itself, every single iteration: if x is
        // present in the whole array, is it still inside [lo, hi]?
        if (std::binary_search(a.begin(), a.end(), x)) {
            bool in_window = false;
            for (int i = lo; i <= hi; ++i)
                if (a[i] == x) { in_window = true; break; }
            if (!in_window) *inv_ever_broken = true;
        }
        ++iters;

        if (a[mid] == x) return mid;
        if (a[mid] < x) lo = mid;           // <-- the one-character bug
        else             hi = mid - 1;
    }
    return -2;                             // stands in for "forever"
}

// --- 3. Ten thousand tests with keys that are present -----
// --- 4. The same ten thousand tests, keys absent -----
// The only change between parts 3 and 4 is the distribution the keys
// are drawn from, and it changes the hang rate from 32% to 100%.

// --- 5. And the invariant? It never broke once. -----
// inv_ever_broken is set by the audit in part 2 and is never set.
// Printing that zero beside the 13,228 hangs IS the experiment: the
// two numbers together say "partially correct, and useless".
printf("    iterations audited : %lld\n", iters);
printf("    invariant violated : %s\n", broken ? "YES" : "NO");
printf("    wrong answers, A+B : %d\n", wrong_present + wrong_absent);

// --- 6. Why proving beats testing, counted -----
// Exhaustive testing means every sorted array and every key. The
// number of sorted arrays of length n over an alphabet of size m is
// the multiset coefficient C(m+n-1, n).
for (int n : {4, 8, 16, 20}) {
    const int m = 100;
    double c = 1.0;
    for (int i = 1; i <= n; ++i) c *= (double)(m + n - i) / i;
    printf("    n=%2d, values 0..99 : %.3g sorted arrays x %d keys\n",
           n, c, m);
}

```

**What to notice.** The invariant violated line prints NO, and it is the most important line in the output: the audit is not failing to find a violation because it is badly written, it is failing because there is none to find.

**Then try to break it.** Change the bug from `lo = mid` to `lo = mid + 2`. Now predict, before running it, which of the three obligations fails — and which column of the table changes. This mutation terminates and gives wrong answers, the exact mirror image of the one in the book, and it is the case an invariant proof catches immediately.

**? Checkpoint — Test Yourself**

1. State a loop invariant for a loop that computes  $\sum_{i=1}^n A[i]$  into a variable *total*, and give the measure that proves it terminates.
2. A classmate's invariant for selection sort reads: "after each iteration,  $A[1..k]$  is sorted". Give an incorrect program that satisfies it, and repair the invariant.
3. Explain, in terms of the three obligations, how a program can be partially correct but not totally correct, and give the measure whose absence causes it.

**✓ Chapter Summary**

- Testing samples an infinite input space. For sorted arrays of 20 two-digit numbers, exhaustive testing needs  $2.5 \times 10^{24}$  cases — about 80 million years. A proof covers every  $n$  at once.
- A loop invariant proof has three obligations: initialisation, maintenance, termination. The first two are induction; the third has no counterpart in induction and is where the real content often lies.
- Initialisation plus maintenance give *partial* correctness — if it halts, it is right. Termination upgrades that to total correctness.
- Insertion sort's invariant must say the prefix holds *the elements originally there*, in order. Drop the permutation clause and the all-zeros "sort" satisfies it.
- Binary search's invariant is about exclusion: if  $x$  is in  $A$  at all, it is in  $A[lo..hi]$ .
- Termination is proved by a loop variant: an integer-valued measure that strictly decreases. Over the reals the argument fails —  $x \leftarrow x/2$ .
- Measured: a binary search with  $lo \leftarrow mid$  gave **zero** wrong answers in 20,000 trials and **zero** invariant violations across 132,311,650 audited iterations — and failed to terminate in 13,228 of those trials. Two obligations out of three is not correctness.

**✍ Review Questions**

1. (MCQ) The three obligations of a loop invariant proof are: (a) initialisation, iteration, conclusion; (b) initialisation, maintenance, termination; (c) precondition, invariant, postcondition; (d) base case, inductive step, generalisation.
2. (MCQ) Initialisation and maintenance alone establish: (a) total correctness; (b) partial correctness; (c) termination; (d) a running-time bound.
3. (Short) Explain why a loop invariant proof is a proof by induction, and identify the one obligation that has no inductive counterpart.
4. (Short) Why must a sorting algorithm's invariant mention that the sorted prefix contains the original elements? Give the program that makes the point.
5. (Short) State the two conditions a loop variant must satisfy, and give an example of a measure satisfying the first but not the second.
6. (Applied) Prove that the following loop computes  $a^n$  for integer  $n \geq 0$ , discharging all three obligations:  $r \leftarrow 1; k \leftarrow n; \text{while } k > 0 \text{ do } r \leftarrow r \cdot a; k \leftarrow k - 1$ .
7. (Applied) The measured experiment found a defect that is never wrong and sometimes never finishes. Construct the mirror-image defect — one that always terminates and is sometimes wrong — in binary search, say which obligation it violates, and describe the test that would find it.

PART II

# Recursion and Divide-and-Conquer

---

*An algorithm that calls itself is easy to write and hard to cost. Part II is the machinery for costing one — three ways to solve a recurrence, a theorem that solves most of them at sight, and the two sorting algorithms that every later chapter uses as a reference point.*



**Figure P2.1:** Reading path through Part II. Chapter 6 is the tool you will reach for most; Chapter 7 is the baseline that makes the rest worth having.

## Recurrences I: Substitution and Recursion Trees

---

Part 1

**Part 2**

Part 3

Part 4

Part 5

Part 6

*Part II — Recursion and Divide-and-Conquer*

### Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

## Recurrences II: The Master Theorem

---

Part 1

**Part 2**

Part 3

Part 4

Part 5

Part 6

*Part II — Recursion and Divide-and-Conquer*

### Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

## Brute Force, and When It Is the Right Answer

---

Part 1

**Part 2**

Part 3

Part 4

Part 5

Part 6

*Part II — Recursion and Divide-and-Conquer*

### Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

## Divide and Conquer: Merge Sort and the Pattern

---

Part 1

**Part 2**

Part 3

Part 4

Part 5

Part 6

*Part II — Recursion and Divide-and-Conquer*

### Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

## Quicksort, Randomisation and the Average Case

---

Part 1

**Part 2**

Part 3

Part 4

Part 5

Part 6

*Part II — Recursion and Divide-and-Conquer*

### Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

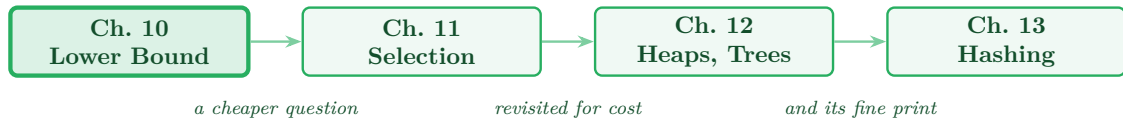
PART III

# Sorting, Selection and the Structures Beneath

---

*Part II costed particular algorithms. Part III asks the harder question — how fast could any algorithm be? — proves an answer for sorting, and then shows exactly what buying an extra assumption gets you.*

*the first result about a problem, not an algorithm*



**Figure P3.1:** Reading path through Part III. Chapter 10 is the hinge of the whole book; Chapters 12 and 13 revisit Data Structures for their analysis, not their implementation.

## The Comparison Lower Bound, and Sorting in Linear Time

---

Part 1

Part 2

**Part 3**

Part 4

Part 5

Part 6

*Part III — Sorting, Selection and the Structures Beneath*

### Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

# Selection and Order Statistics

---

|        |        |               |        |        |        |
|--------|--------|---------------|--------|--------|--------|
| Part 1 | Part 2 | <b>Part 3</b> | Part 4 | Part 5 | Part 6 |
|--------|--------|---------------|--------|--------|--------|

*Part III — Sorting, Selection and the Structures Beneath*

 **Not written yet**

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

## Heaps and Search Trees, Revisited for Analysis

---

Part 1

Part 2

**Part 3**

Part 4

Part 5

Part 6

*Part III — Sorting, Selection and the Structures Beneath*

### Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

## Hashing, and What Constant Time Actually Promises

---

Part 1

Part 2

**Part 3**

Part 4

Part 5

Part 6

*Part III — Sorting, Selection and the Structures Beneath*

### Not written yet

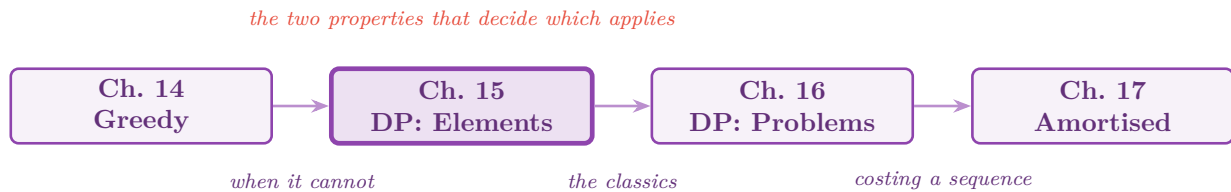
This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

PART IV

# The Design Paradigms

---

*Divide-and-conquer splits a problem into independent pieces. Part IV is what to do when the pieces are not independent — when a local choice can be trusted, when it cannot and the subproblems overlap, and how to cost a sequence of operations rather than one.*



**Figure P4.1:** Reading path through Part IV. Chapter 15 is the one to understand rather than memorise: the two properties in it decide whether greedy or dynamic programming applies at all.

## Greedy Algorithms

---

Part 1

Part 2

Part 3

**Part 4**

Part 5

Part 6

*Part IV — The Design Paradigms*

### Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

## Dynamic Programming I: The Elements

---

Part 1

Part 2

Part 3

**Part 4**

Part 5

Part 6

*Part IV — The Design Paradigms*

### Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

## Dynamic Programming II: Classic Problems

---

Part 1

Part 2

Part 3

**Part 4**

Part 5

Part 6

*Part IV — The Design Paradigms*

### Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

# Amortised Analysis

---

 Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

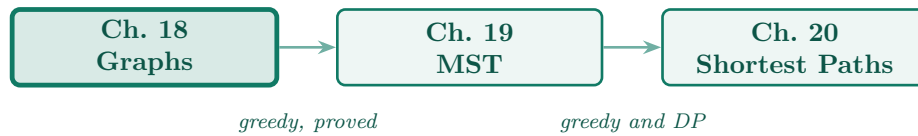
PART V

# Graph Algorithms

---

*Everything so far has been applied to arrays. Part V applies it to graphs, where one decision — how the graph is stored — changes every bound in the part, and where the design paradigms of Part IV reappear wearing different clothes.*

*sparsity changes every bound that follows*



**Figure P5.1:** Reading path through Part V. Chapter 18 comes first because the representation decides the bounds: the same algorithm is a different cost on a sparse graph and a dense one.

## Graphs: Representation, Traversal and Sparsity

---

Part 1

Part 2

Part 3

Part 4

**Part 5**

Part 6

*Part V — Graph Algorithms*

### Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

## Minimum Spanning Trees

---

Part 1

Part 2

Part 3

Part 4

**Part 5**

Part 6

*Part V — Graph Algorithms*

### Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

## Shortest Paths: Single-Source and All-Pairs

---

Part 1

Part 2

Part 3

Part 4

**Part 5**

Part 6

*Part V — Graph Algorithms*

### Not written yet

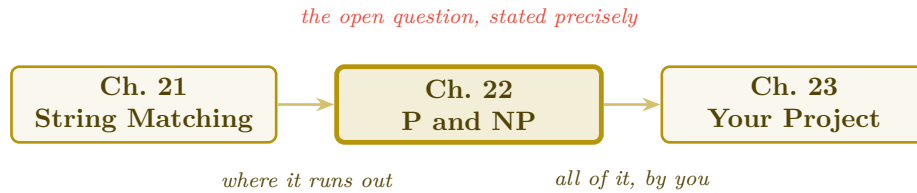
This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

PART VI

# Strings, Limits and Practice

---

*The book closes where the analysis runs out. Chapter 21 is one more well-solved problem; Chapter 22 is the class of problems nobody can solve efficiently and nobody can prove cannot be — the largest open question in the subject, stated precisely enough to recognise.*



**Figure P6.1:** Reading path through Part VI. Chapter 22 is the only chapter whose central claim is that nobody knows; Chapter 23 is where you use the rest.

# String Matching

---

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

*Part VI — Strings, Limits and Practice*

 Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

## Complexity Classes: P, NP and NP-Completeness

---

Part 1

Part 2

Part 3

Part 4

Part 5

**Part 6**

*Part VI — Strings, Limits and Practice*

### Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

# The Semester Project

---

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

*Part VI — Strings, Limits and Practice*

 Not written yet

This chapter is a placeholder. It exists so that the master document builds with no undefined references while the chapters are written one at a time.

# Appendices

---

## The C++ Toolkit, and How to Time Honestly

---

 Not written yet

This appendix is a placeholder.

## The Mathematics This Course Assumes

---

 Not written yet

This appendix is a placeholder.

## Answers to the Checkpoints

---

 Not written yet

This appendix is a placeholder.

## Glossary

---

 Not written yet

This appendix is a placeholder.

## Resources and Further Reading

---

 Not written yet

This appendix is a placeholder.

## The Boundary with the Neighbouring Courses

---

 Not written yet

This appendix is a placeholder.

## The Sixteen-Week Schedule

This appendix is the **source of record** for the teaching schedule. The one-page `AoA_Fall2026_Lecture_Schedule.pdf` distributed to students is generated from it, and the slide pipeline parses the table below. Change this first; regenerate the one-page version afterwards.

Twenty-three chapters are taught across sixteen weeks, so seven weeks carry two chapters. Those pairings are not arbitrary: in every case the second chapter is the one that cannot be stated honestly without the first —  $O$  and  $\Omega$  with  $\Theta$ , merge sort with the divide-and-conquer pattern that produces it, graphs with the representation that decides their cost.

### G.1 The Schedule

| Wk | Lecture topic                                        | Chapters | Lab                                                                  | Assessment              |
|----|------------------------------------------------------|----------|----------------------------------------------------------------------|-------------------------|
| 1  | What an algorithm is; input size and the three cases | 1, 2     | Growth measured: the ratio test, and the crossover at $n \approx 65$ | —                       |
| 2  | Asymptotic notation, formally                        | 3        | Fit a curve; exclude the exponents either side                       | —                       |
| 3  | Correctness and loop invariants                      | 4        | A bug that is never wrong and never finishes                         | —                       |
| 4  | Recurrences; the master theorem                      | 5, 6     | Recursion trees against measured depth and cost                      | —                       |
| 5  | Brute force; divide and conquer                      | 7, 8     | Where $n^2$ beats $n \lg n$ , and by how much                        | —                       |
| 6  | Quicksort and randomisation                          | 9        | The adversarial input, and what randomising costs                    | <b>Quiz 1 (Ch 1–6)</b>  |
| 7  | The comparison lower bound; linear-time sorting      | 10       | Counting and radix sort against <code>std::sort</code>               | <b>Assignment 1 set</b> |
| 8  | Selection and order statistics                       | 11       | Midterm, then quickselect against a full sort                        | <b>Midterm (Ch 1–9)</b> |
| 9  | Heaps, search trees and hashing, analysed            | 12, 13   | A hash table degrading under a poor hash                             | <b>Assignment 1 due</b> |
| 10 | Greedy algorithms and exchange arguments             | 14       | Huffman coding on real text; measure the compression                 | —                       |
| 11 | Dynamic programming: elements and classics           | 15, 16   | Memoised against bottom-up, timed and counted                        | —                       |

| Wk | Lecture topic                                               | Chapters | Lab                                                           | Assessment                                    |
|----|-------------------------------------------------------------|----------|---------------------------------------------------------------|-----------------------------------------------|
| 12 | Amortised analysis                                          | 17       | A growing array:<br>aggregate cost against<br>worst-case cost | <b>Quiz 2 (Ch 10–16)</b>                      |
| 13 | Graph<br>representation,<br>traversal and<br>spanning trees | 18, 19   | The matrix/list crossover<br>as density falls                 | <b>Assignment 2 set;<br/>project proposal</b> |
| 14 | Shortest paths                                              | 20       | Dijkstra with and<br>without a priority queue                 | —                                             |
| 15 | String matching;<br>P, NP and<br>NP-completeness            | 21, 22   | Naive, Rabin–Karp and<br>KMP on the same corpus               | <b>Assignment 2 due</b>                       |
| 16 | The semester<br>project                                     | 23       | Implement, measure,<br>defend                                 | <b>Project due; Final<br/>(Ch 1–22)</b>       |

## G.2 How the Weeks Map to the HEC Outline

Every clause of the published HEC 2023 outline is reached, and the Course Specification in the front matter traces each clause to its chapter. The table below is the same trace read the other way round — by week, for a course file that asks when a topic is delivered rather than where it is written.

| Outline clause                                                      | Weeks | Note                                                            |
|---------------------------------------------------------------------|-------|-----------------------------------------------------------------|
| Introduction; role<br>of algorithms in<br>computing                 | 1     | Ch 1                                                            |
| Analysis on nature of<br>input and size of input                    | 1     | Ch 2; CLO-1 and CLO-2                                           |
| Asymptotic notations;<br>$O$ , $\Omega$ , $\Theta$ , $o$ , $\omega$ | 2     | Ch 3; CLO-5                                                     |
| Sorting algorithm<br>analysis                                       | 5–8   | Ch 8–11, including the lower bound the<br>outline does not name |
| Loop invariants                                                     | 3     | Ch 4                                                            |
| Recursion and<br>recurrence relations                               | 4     | Ch 5, 6                                                         |
| Algorithm design<br>techniques; brute force                         | 5     | Ch 7                                                            |
| Divide-and-conquer;<br>merge, quick sort                            | 5, 6  | Ch 8, 9; CLO-6                                                  |
| Greedy approach                                                     | 10    | Ch 14                                                           |
| Dynamic programming;<br>elements of DP                              | 11    | Ch 15, 16                                                       |
| Search trees; heaps                                                 | 9     | Ch 12 — analysed, not reimplemented                             |
| Hashing                                                             | 9     | Ch 13                                                           |

| Outline clause                                        | Weeks  | Note            |
|-------------------------------------------------------|--------|-----------------|
| Graph algorithms;<br>shortest paths; sparse<br>graphs | 13, 14 | Ch 18–20; CLO-7 |
| String matching                                       | 15     | Ch 21; CLO-8    |
| Introduction to<br>complexity classes                 | 15     | Ch 22; CLO-4    |

💡 Two weeks the outline does not ask for

**Week 7 (Chapter 10), the comparison lower bound.** The outline says “sorting algorithm analysis” and lists the sorts. It does not ask for the  $\Omega(n \lg n)$  decision-tree proof. It is taught anyway because it is the first result in the course about a *problem* rather than an algorithm, and because without it “merge sort is optimal” is an assertion rather than a theorem.

**Week 12 (Chapter 17), amortised analysis.** Also unnamed. Chapter 13’s claim about hash tables and Chapter 19’s union-find both depend on it, and CLO-3 asks for the time complexity of simple algorithms — which, for a sequence of operations on a growing structure, is an amortised question. The front matter states this justification in full.

⚠️ If the semester is shorter than sixteen weeks

Cut in this order, and say so in the course file:

1. **Chapter 16** (dynamic programming problems) down to two problems rather than five. Chapter 15 carries the examinable content.
2. **Chapter 21** (string matching) down to naive and Rabin–Karp, leaving KMP’s prefix function as reading. CLO-8 asks only that a string-matching algorithm be traced or implemented.
3. **Chapter 12** to a single lecture, since Data Structures has already built these structures and this chapter only re-analyses them.

Do *not* cut Chapters 3, 4, 6, 10 or 15. Each is a prerequisite for something later, and Chapter 10 is the only place the course proves a lower bound at all.